

ID:

**Laboratório de Sistemas Operacionais
Implementação de um Sistema Operacional
Baseado em uma Arquitetura MIPS**

São José dos Campos - Brasil

Dezembro de 2018

ID:

Laboratório de Sistemas Operacionais

Implementação de um Sistema Operacional Baseado em uma Arquitetura MIPS

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Sistemas Operacionais.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Dezembro de 2018

Resumo

Este relatório tratou da implementação de um sistema operacional compatível com a arquitetura previamente desenvolvida e seu respectivo compilador para linguagem C-. Assim, realizou-se o desenvolvimento dos módulos do processador em Verilog para sustentar o SO assim como as mudanças necessárias no compilador e no conjunto de instruções e o código em C- do SO, mudanças que seguiram o que foi proposto no relatório anterior.

Palavras-chaves: *SO. E/S.* Gerenciamento. Arquitetura. MIPS. Contexto.

Lista de Algoritmos

4.1	Algoritmo Exemplo:C-	20
4.2	Algoritmo Exemplo: <i>Assembly</i>	21
4.3	Módulo BIOS em Verilog	25
4.4	Módulo HD em Verilog	26
4.5	Módulo <i>Buffer Bank</i> em Verilog	28
4.6	Módulo <i>Interruption Controller</i> em Verilog	29
4.7	Módulo <i>Timer</i> de Interrupção em Verilog	31
4.8	BIOS em C-	33
4.9	Variáveis de Controle de Arquivos	36
4.10	Variáveis de Controle dos Menus LCD	36
4.11	<i>Prompt</i> de Comando	37
4.12	Carregando um Programa	39
4.13	Rotina de Escrita de Arquivos	40
4.14	Rotina de Renomeação de Arquivos	43
4.15	Rotina para Deletar Arquivos	44
4.16	Rotina de Escalonamento de Programas	45
4.17	Rotina de Entrada de Dados	50
4.18	Rotina de Saída de Dados	51
A.1	Código do SO em C-	69

Lista de ilustrações

Figura 1 – Interfaces do Sistema	10
Figura 2 – Estados dos Processos	11
Figura 3 – Estruturação de um Arquivo	15
Figura 4 – Estruturação de Diretórios em Nível Único	15
Figura 5 – Estruturação de Diretórios em Nível Hierárquico	16
Figura 6 – Comunicação e Barramento E/S	17
Figura 7 – Funcionamento de uma DMA	18
Figura 8 – Gramática BNF para C-	20
Figura 9 – Teste BIOS - 1	53
Figura 10 – Teste BIOS - 2	54
Figura 11 – Teste BIOS - 3	54
Figura 12 – Teste BIOS - 4	55
Figura 13 – Teste BIOS - 5	55
Figura 14 – <i>Prompt</i> do SO	56
Figura 15 – Escalonador de Processos - 1	57
Figura 16 – Escalonador de Processos - 2	57
Figura 17 – Escalonador de Processos - 3	58
Figura 18 – Escalonador de Processos - 4	58
Figura 19 – Escalonador de Processos - 5	59
Figura 20 – Escalonador de Processos - 6	59
Figura 21 – Escalonador de Processos - 7	60
Figura 22 – Escalonador de Processos - 8	60
Figura 23 – Escalonador de Processos - 9	61
Figura 24 – Escalonador de Processos - 10	61
Figura 25 – Deletar Arquivos - 1	62
Figura 26 – Deletar Arquivos - 2	63
Figura 27 – Escrever Arquivos - 1	64
Figura 28 – Escrever Arquivos - 2	64
Figura 29 – Escrever Arquivos - 4	65

Lista de tabelas

Tabela 1 – Conjunto de Instruções	22
Tabela 1 – Conjunto de Instruções	23

Sumário

1	INTRODUÇÃO	8
2	OBJETIVOS	9
2.1	Geral	9
2.2	Específico	9
3	FUNDAMENTAÇÃO TEÓRICA	10
3.1	Gerenciamento de Processos	11
3.2	Gerenciamento de Memória	13
3.2.1	Espaços de Endereçamento	13
3.2.2	Técnicas de Gerenciamento	14
3.3	Gerenciamento de Arquivos	15
3.4	Gerenciamento de Entrada e Saída	16
3.5	BIOS	18
4	DESENVOLVIMENTO	19
4.1	Inclusão de <i>Software</i> e <i>Hardware</i> em Laboratórios Anteriores	19
4.1.1	<i>Software</i>	19
4.1.2	Arquitetura	21
4.2	Mudanças na Arquitetura	22
4.2.1	BIOS	25
4.2.2	HD	26
4.2.3	<i>Buffer Bank</i>	28
4.2.4	<i>Interrupton Controller</i>	29
4.2.5	<i>Timer</i> de Interrupção	31
4.2.6	Saída de Dados	33
4.3	BIOS em C-	33
4.4	Lógica e Implementação do SO em C-	34
4.4.1	Funções do Sistema	34
4.4.2	Inicialização do Sistema	36
4.4.3	<i>Prompt</i> de Comando	37
4.4.4	Gerenciamento de Arquivos	38
4.4.4.1	Carregamento de Arquivos/Programas	39
4.4.4.2	Escrita de Arquivos	40
4.4.4.3	Renomeação de Arquivos	43
4.4.4.4	Deleção de Arquivos	44

4.4.5	Gerenciamento de Processos e Memória	45
4.4.5.1	Interrupção por Estouro de <i>Quantum</i>	49
4.4.5.2	Interrupção por E/S	50
4.4.5.3	Interrupção por Fim de Processo	51
5	RESULTADOS OBTIDOS	53
6	CONSIDERAÇÕES FINAIS	66
	REFERÊNCIAS	67
	APÊNDICES	68
	APÊNDICE A – APÊNDICE 1	69

1 Introdução

O presente relatório caracteriza-se como o segundo da série, feito com a finalidade de relatar a implementação sistema operacional para a arquitetura MIPS e compilador para C- projetados nos laboratórios anteriores, definindo-se os módulos em Verilog, as instruções adicionadas para o correto funcionamento, as mudanças na arquitetura e o código em C- do Sistema Operacional.

Um computador é um sistema capaz de executar diversas instruções com base em um *hardware* e arquitetura, sendo útil para cálculos, aplicações de usuários, navegar na *web* e diversas outras atividades. Para isso, um conjunto de módulos de *hardware* e um conjunto de instruções compatível são necessários, além de uma linguagem de alto nível para desenvolvimento de aplicações, capaz de existir por conta de um compilador.

Além disto, este sistema deve ser gerenciado para que seus recursos possam ser usados de forma correta e coerente, disponibilizando uma plataforma para o desenvolvimento de aplicações e programas que possam utilizar dos recursos de *hardware* e da *arquitetura* disponíveis. Assim, um usuário não necessita saber como controlar todos os componentes de um computador e muito menos gerenciá-lo, outrora o mesmo não conseguiria desenvolver um código muito extenso e útil sem ter que gastar horas para o gerenciamento dos componentes e recursos.

Os sistemas operacionais surgiram então para "facilitar" a vida do usuário, sendo estes responsáveis pelo gerenciamento de recurso e dispositivos de *hardware*, fornecendo uma plataforma de desenvolvimento consolidada para o usuário final que agora pode desenvolver programas e aplicações sem o fardo de saber como utilizar todos os recursos providos pelo *hardware* a fundo. Portanto, um SO não passa de um *software* que age como uma interface entre o usuário e o *hardware* do computador.

No desenvolvimento deste trabalho, abordou-se o escopo da implementação de um sistema operacional, definindo-se os módulos adicionados ao processador, as instruções novas, as mudanças no compilador para C- e como o código em C- foi estruturado. Assim buscou-se explicar como a integração entre arquitetura, compilador e SO foi feita, colocando em prática os conhecimentos em arquitetura de computadores, sistemas operacionais, compiladores e linguagem de descrição de *hardware* Verilog, com a utilização do *software* Quartus e da placa FPGA.

2 Objetivos

2.1 Geral

Com o desenvolvimento deste projeto, buscou-se a implementação de um sistema operacional que assemelha-se a um SO capaz de realizar todas as operações básicas necessárias para que este possa gerenciar o processador MIPS já desenvolvido em conjunto com o compilador C-, realizando testes posteriores na FPGA. Com isso, buscou-se também adquirir conhecimento sobre a implementação de um sistema computacional mais complexo e que coloca-se os conhecimentos em sistemas operacionais em prática.

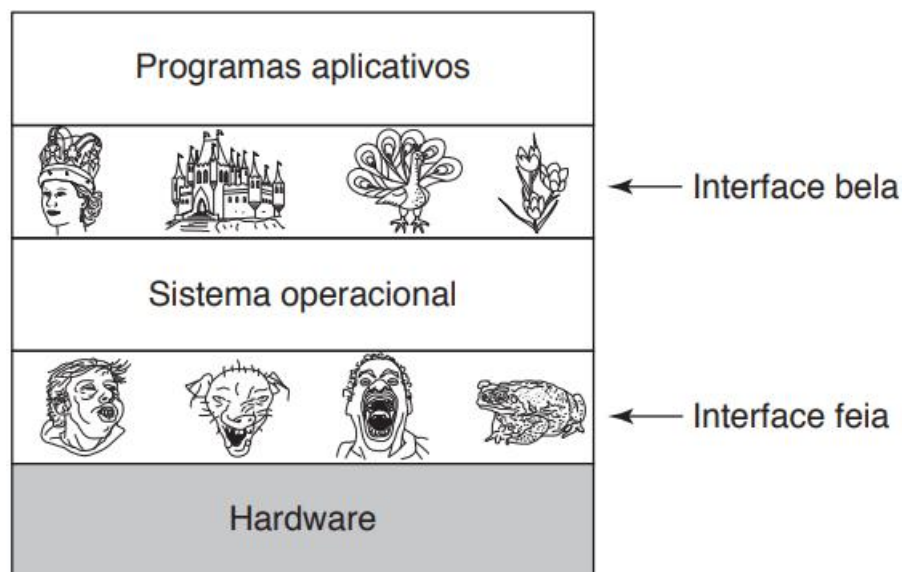
2.2 Específico

Neste projeto, buscou-se a implementação dos módulos em Verilog projetados no relatório anterior, criação de instruções para suportar o SO e o código do sistema operacional em si, com o objetivo de serem testados na placa FPGA.

3 Fundamentação Teórica

Um sistema operacional é um *software* responsável por fornecer aos programadores um conjunto de recursos abstratos ao invés de recursos complexos de *hardware* e gerenciar todos os recursos de *hardware*[1]. Lidar com discos rígidos no nível de *hardware* por exemplo é uma tarefa extensa, complexa e impraticável para um programador que busca desenvolver um *software* mais complexo, neste momento o SO entra em ação, onde este implemente um *driver* capaz de fornecer uma interface de uso mais simples ao usuário, uma abstração necessária, e são boas abstrações dos recursos de *hardware* que tornam um SO tão importante[1]. O SO então acaba por ser um mediador entre o *hardware* e o programador, facilitando o *design* de *softwares* em alto nível, como visto na Figura 1, onde o *hardware* complexo é intermediado pelo SO para que o programador possa criar *softwares* com menos complicações.

Figura 1 – Interfaces do Sistema



Fonte: Sistemas Operacionais Modernos[1]

Além disso, os sistemas operacionais podem ser vistos como gerenciadores de recursos numa visão *bottom-up*[1], onde estes são responsáveis por gerenciarem a memória do sistema, alocando corretamente espaços de memória e realizando endereçamentos corretos, gerenciam a entrada e saída do sistema, alocando dispositivos e controlando a execução da leitura e impressão de dados, gerenciam os processos, endereçando cada qual com seus recursos respectivos e gerenciam arquivos, mantendo uma organização das pastas e arquivos armazenados nos sistemas e as permissões de acesso de cada[2].

3.1 Gerenciamento de Processos

Um sistema operacional gerencia diversos processos que executam instruções diferentes e realizam tarefas diferentes. Uma aplicação pode ter diversos processos associados à ela, sendo que cada processo é responsável por realizar uma tarefa importante à aplicação.

Um processo é caracterizado por ser uma instância de um programa em execução[1], armazenando o contador de programa atual, os registradores, as variáveis e o espaço de endereçamento. O papel do SO neste caso é alocar o espaço de memória ao processo, manter o número do contador de programa e gerenciar as variáveis e registradores e, além disto, chavear entre diversos processos a serem executados.

Como um processo lida com processar dados de entrada, gerar sinais de saída e executar diversas instruções, este pode acabar por passar períodos de tempo em ociosidade, esperando por entrada ou saída por exemplo, podendo assim serem escalonados enquanto não executam nenhuma instrução para que outros processos possam ser executados durante esta espera[1]. Para isso, o sistema operacional deve ser capaz de realizar uma troca de contexto, salvando os dados atuais do processo(contador de programa, variáveis, registradores) e iniciar outro processo do começo ou do meio, caso este tenha sido escalonado em algum momento. Este tipo de paradigma caracteriza um sistema de multiprogramação, sendo este sistema capaz de gerar a impressão de que um computador com apenas um processador esteja executando diversos processos ao mesmo tempo[2].

Os processos então possuem diversos estados e transições que caracterizam seu comportamento em um sistema de multiprogramação, vistos na [Figura 2](#).

Figura 2 – Estados dos Processos



1. O processo é bloqueado aguardando uma entrada
2. O escalonador seleciona outro processo
3. O escalonador seleciona esse processo
4. A entrada torna-se disponível

Fonte: Sistemas Operacionais Modernos[1]

Durante uma troca de contexto, o processo a ser interrompido está em execução e passa para o estado pronto, onde aguarda ser escalonado novamente. Caso o processo esteja bloqueado à espera de entrada ou por algum outro motivo, ele mantém-se bloqueado

até que a entrada esteja pronta, passando-se assim para o estado pronto onde pode ser escalonado novamente para ser executado.

Além da capacidade da troca de contexto, o SO deve implementar algum tipo de algoritmo de escalonamento para melhor gerenciamento do tempo de execução dos processos e o momento certo para se trocar de contexto. Existem diversos tipos de algoritmos para escalonamento, sendo que cada um pode ser útil para uma situação diferente, sendo que existem algoritmos para sistemas em lote, interativos e em tempo real. Dentre estes existem[1]:

1. Lote:

- a) **Primeiro a chegar, primeiro a ser servido:** as tarefas chegam em filas e são executadas por ordem de chegada, sem preempção, ou seja, uma tarefa executa quanto quiser. Quando o processo é bloqueado, o próximo da fila executa. Ao passar para pronto, volta ao fim da fila.
- b) **Tarefa mais curta primeiro:** Os processos chegam na fila de execução e o processo com menor tempo de conclusão é executado primeiro.
- c) **Tempo restante mais curto em seguida:** Os processos chegam na fila de execução e a tarefa com menor tempo de execução restante é escolhida. Caso uma tarefa nova tenha menor tempo de conclusão que o processo atual, ele é escalonado para esta nova tarefa.

2. Interativo:

- a) **Chaveamento circular (*Round-Robin*):** Os processos chegam na lista de execução e cada um possui um *quantum* (tempo de execução). Caso este tempo acabe, o processo é chaveado e o próximo na fila é executado. Caso seja bloqueado antes de seu *quantum* acabar, é chaveado para o próximo da fila. Ao ser chaveado, um processo é colocado no fim da lista.
- b) **Prioridades:** cada processo possui uma prioridade de execução no sistema, sendo aqueles com prioridades mais altas executados primeiros. Pode-se diminuir a prioridade de um processo com o tempo para outros executarem ou atribuir um *quantum* para escalonamento.
- c) **Processo mais curto primeiro:** executar o processo dito como mais curto primeiro. Para isso, utiliza-se da técnica de envelhecimento, onde estima-se o tempo de execução de um processo com base na média ponderada do valor atual e a estimativa anterior.
- d) **Loteria:** distribuir bilhetes de loteria que significam uma fração do tempo de execução na CPU. Processos mais importantes recebem mais bilhetes, consequentemente possuindo mais tempo de CPU.

- e) **Fração justa:** mesmo que um usuário possua 9 processos e outro 1, ambos podem possuir 50% do tempo de CPU, diferente do algoritmo *Round-Robin*, onde o primeiro usuário teria 90% de CPU.

3. Tempo Real:

- a) **Crítico:** Existem prazos a serem cumpridos, ou seja, um processo deve executar determinada tarefa em um tempo restrito, não há tolerância à atrasos.
- b) **Não crítico:** O descumprimento de prazos é tolerável.

3.2 Gerenciamento de Memória

Todo os computadores possuem hierarquias de memória importantes para a organização estrutural do sistema de memória. O uso de apenas um tipo de memória é inviável, pois existem múltiplos cenários em que diferentes tipos de memória são necessários, sendo que cada tipo de memória tem um custo e velocidade associados. Portanto, utilizar apenas memória barata e mais lenta fica inviável para o desempenho geral do computador, assim como apenas memória rápida e cara inviabiliza os custos de produção[3].

Assim, a hierarquia de memória é utilizada para definir-se os diferentes usos e custos de diferentes tipos de memória. Por exemplo, os dados permanentes de um computador são escritos em uma memória barata e lenta, pois a sua quantidade pode ser maior, entretanto a memória principal precisa ser mais rápida para melhores tempos de execução, portanto uma memória mais rápida e mais cara deve ser utilizada[3].

Em conjunto com a hierarquia de memória, deve-se ter um gerenciamento adequado administrado pelo sistema operacional, capaz de endereçar corretamente os espaços de memória, alocar e desalocar memória aos processos no contexto e, de forma geral, otimizar o uso deste recurso para maior desempenho sempre[1].

3.2.1 Espaços de Endereçamento

A memória física não serve como um endereçamento otimizado por si só, acessando-se posições físicas diretas. Para isso, criou-se os espaços de endereçamento[1]. Cada processo possui seu próprio espaço de endereçamento separado de todos os outros, sendo que as posições de endereçamento locais são convertidas em espaços físicos da memória principal.

Esta abordagem soluciona o problema de vários processos carregados na memória ao mesmo tempo, já que cada um não interfere na zona de endereçamento do outro e também evita problemas de segurança, onde um processo poderia escrever numa zona de memória não pertencente a ele.

Algumas técnicas foram desenvolvidas para se gerenciar melhor esta divisão da memória, sendo estas o *swapping* e a memória virtual. A técnica de *swapping* lida com o problema do tamanho máximo da memória, onde muitos processos não podem coexistir nela ao mesmo tempo, assim o estado de um processo é salvo no disco rígido para que seja escalonado em outro momento[1]. A técnica de memória virtual busca deixar processos parcialmente carregados na memória e dados adicionais salvos no disco rígido, estendendo a memória principal para o disco rígido.

3.2.2 Técnicas de Gerenciamento

Diversas técnicas existem para o gerenciamento de memória livre (dinâmica), dentre elas estão[1]:

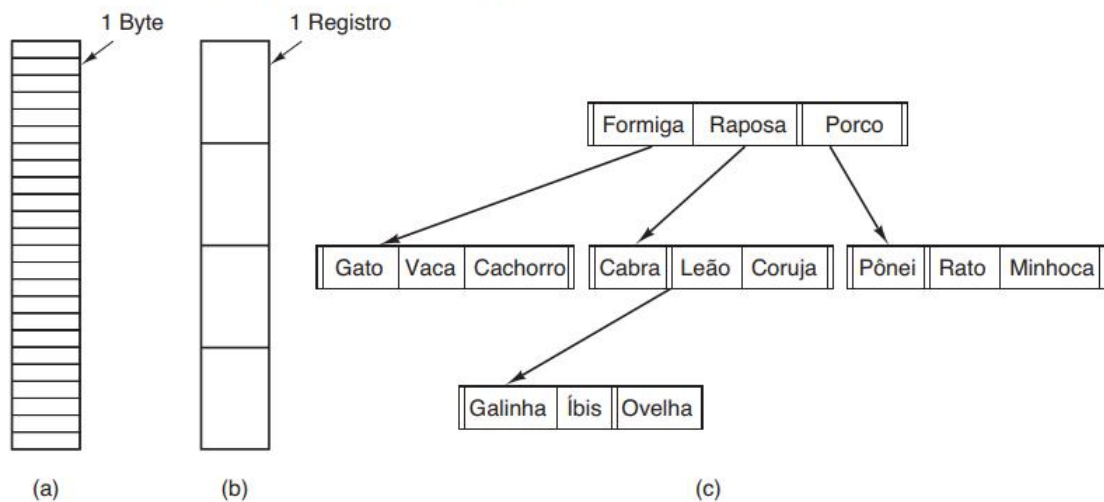
- **Mapa de *Bits*:** a memória é dividida em segmentos vaiados que podem ser muito pequenos até alguns quilobytes, sendo que cada segmento possui um *bit* que informa se está livre ou não.
- **Lista Encadeada:** lista encadeada de espaços livres e de segmentos de memória alocados.
- **Paginação:** os segmentos de memória são divididos e endereçados por meio de endereços virtuais que são convertidos em endereços reais pela MMU. Com isso, o sistema operacional pode gerenciar blocos uniformes chamados de páginas, sendo que estas páginas podem estar na memória principal como no disco rígido. Isto implica que o SO pode carregar programas de maior tamanho do que a memória principal e fazer a troca de páginas caso necessário para o funcionamento correto do processo. A troca de página ocorre quando existe um *page fault*, sendo esta buscada na memória secundária e inserida na memória principal. Dentro do escopo de paginação, existem diversas técnicas de substituição de página para maior desempenho, sendo estas:
 - Não Usadas Recentemente(NRU);
 - Primeiro a Entrar, Primeiro a Sair(FIFO);
 - Segunda Chance;
 - Relógio;
 - Usadas Menos Recentemente(LRU);
 - Conjunto de Trabalho;
 - *WSClock*;

3.3 Gerenciamento de Arquivos

Arquivos não passam de abstrações geradas pelo sistema operacional e, de forma mais simples, são dados permanentes escritos ou lidos por processos. Arquivos então podem ser estruturados de diversas maneiras, sendo estas diferentes dependendo da implementação utilizada no sistema operacional[1], como visto na Figura 3.

Figura 3 – Estruturação de um Arquivo

Três tipos de arquivos. (a) Sequência de bytes. (b) Sequência de registros. (c) Árvore.

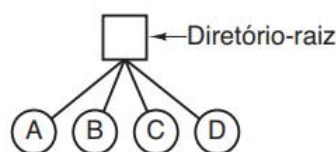


Fonte: Sistemas Operacionais Modernos[1]

Além disto, um sistema de arquivos inclui diretórios que podem ser estruturados para conter pastas e subpastas, que por si só também são arquivos, mas podem conter diversos arquivos e pastas. Arquivos também possuem diversos tipos e formatações. Num sistema operacional atual por exemplo, existem arquivos do tipo texto(.txt), arquivos de música(.mp3), arquivos *web*(.html) e assim por diante. A designação de um tipo diferente de arquivo determina como este será interpretado pelo SO e como seus dados serão guardados[1].

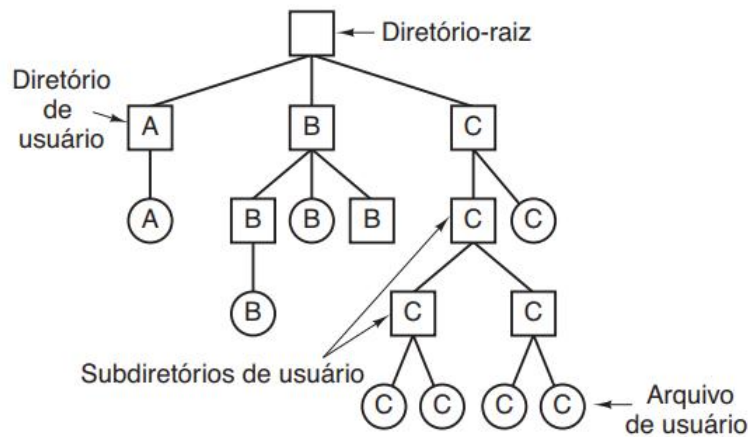
Diretórios também possuem dois tipos de organização: nível único ou hierárquico, vistos na Figura 4 e Figura 5 respectivamente.

Figura 4 – Estruturação de Diretórios em Nível Único



Fonte: Sistemas Operacionais Modernos[1]

Figura 5 – Estruturação de Diretórios em Nível Hierárquico



Fonte: Sistemas Operacionais Modernos[1]

Por fim, o sistema de arquivos pode ser implementado seguindo diferentes modos, sendo estes:

1. **Alocação Contígua:** arquivos armazenados em uma alocação contígua de blocos no disco rígido. Um arquivo de 50KB num sistema com blocos de 5KB ocuparia 10 blocos seguidos de dados. Entretanto, a fragmentação do disco é muito grande.
2. **Lista Encadeada:** o arquivo é alocado em forma de blocos que formam uma lista encadeada, sendo que cada bloco aponta para o outro, onde apenas uma parte dos dados é para apontamento e o resto para dados. O problema de fragmentação é solucionado, porém o acesso aos dados é bem mais lento.
3. **Lista Encadeada com Tabela na Memória:** da mesma maneira que o anterior, porém os apontamentos para os blocos estão salvos numa tabela na memória principal, aumentando a quantidade de dados a serem salvos e melhorando o desempenho de acesso a dados.
4. **I-Nodes:** cada arquivo possui uma estrutura de dados contendo informações sobre o arquivo e onde encontrar seus blocos de dados em disco. Uma vantagem é que os dados de um arquivo só ficam na memória caso este esteja aberto, salvando espaço de memória principal.

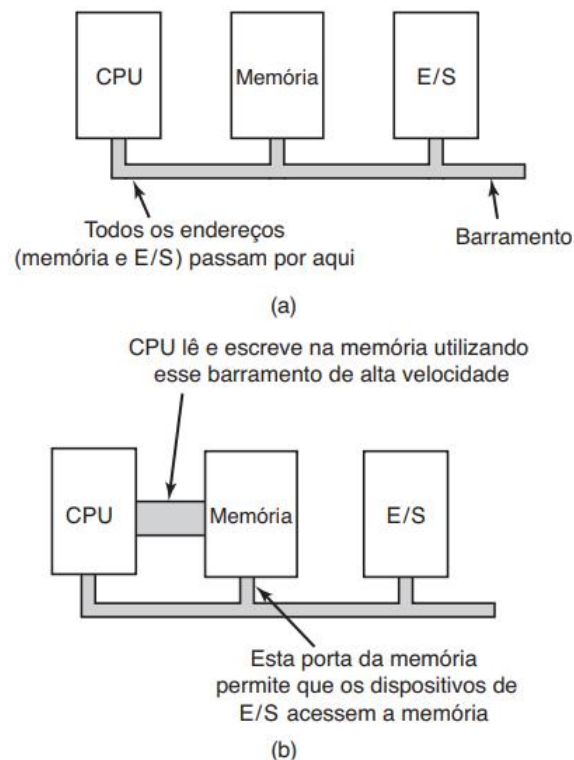
3.4 Gerenciamento de Entrada e Saída

O sistema operacional deve ser responsável por lidar com todos os dispositivos de entrada e saída, assim como tratar todas as exceções e interrupções relacionadas. Estes

dispositivos tem a característica de serem bem mais lentos do que o resto do sistema computacional, por isso precisam de um tratamento especial[2].

Assim, a comunicação entre CPU, memória e dispositivos E/S deve ser projetada e gerenciada pelo SO de forma a otimizar a troca de dados. Para isto, existem dois projetos vistos na Figura 6.

Figura 6 – Comunicação e Barramento E/S

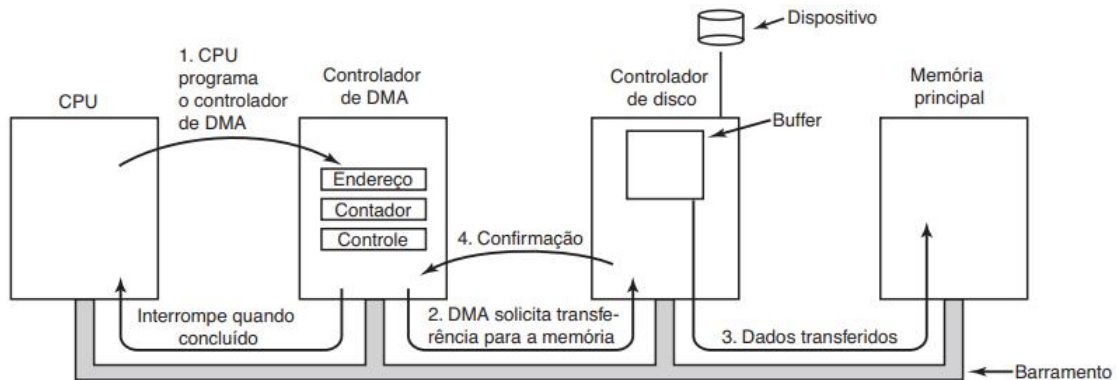


Fonte: Sistemas Operacionais Modernos[1]

São dois métodos que acabam por diminuir a eficiência do uso de E/S por fazerem a CPU esperar o fim de uma operação de E/S e por ocuparem o barramento enquanto realizam a troca de dados.

Para isso, desenvolveu-se a DMA(*Direct Memory Access*), dispositivo capaz de mediar o controle e sinalizações entre os dispositivos de E/S e a CPU. Assim, a CPU é capaz de sinalizar o uso de E/S e continuar a trabalhar enquanto a DMA cuida de manter os sinais de interrupção e sinalizar a CPU quando pronto e ao mesmo tempo controlar a escrita em memória. Este esquema pode ser visto na Figura 7

Figura 7 – Funcionamento de uma DMA



Fonte: Sistemas Operacionais Modernos[1]

A técnica que utiliza a espera de sinais e bloqueia o processamento até que a entrada esteja pronta é a E/S programada.

3.5 BIOS

A BIOS do sistema, também conhecida como *Basic Input/Output System*, é um sistema iniciado ao ligar-se o computador, responsável por realizar testes no sistema, como integridade dos componentes e depois carrega o sistema operacional a partir de uma partição principal no HD, geralmente o MBR(*Master Boot Record*)[4].

A BIOS encontra-se numa memória ROM presente, geralmente, na placa mãe do sistema. Por questões de segurança, a BIOS não pode ser modificada pelo usuário, por isso encontra-se em uma memória ROM protegida de acessos externos[4].

4 Desenvolvimento

Para o desenvolvimento deste projeto, precisou-se ajustar a arquitetura MIPS previamente construída e adicionar detalhes ao compilador C- também desenvolvido anteriormente.

Por conseguinte, o sistema operacional foi desenvolvido na linguagem alvo do compilador, C- e todas as alterações na arquitetura foram feitas em Verilog.

4.1 Inclusão de *Software* e *Hardware* em Laboratórios Anteriores

4.1.1 *Software*

Para o funcionamento do sistema operacional, desenvolveu-se no Laboratório de Compiladores um compilador para a linguagem de programação de alto nível conhecida como C-. Este compilador é capaz de transformar um código em linguagem C- para a linguagem *assembly* correspondente da arquitetura desenvolvida que por sua vez é transformada na linguagem binária respectiva.

A linguagem C- prevê suporte a variáveis do tipo inteiro, incluindo vetores, funções *void* e *int*, chamadas de funções, laços de repetição, estruturas condicionais, retorno de funções, recursão e passagem de dados por parâmetros.

O compilador foi desenvolvido em linguagem C utilizando os pacotes *flex* e *bison*. O compilador segue as regras descritas na [Figura 8](#), sendo que este é capaz de analisar, verificar e reportar erros léxicos, sintáticos e semânticos antes da geração do binário e *assembly*.

Figura 8 – Gramática BNF para C-

```

programa → declaração-lista
declaração-lista → declaração-lista declaração | declaração
declaração → var-declaração | fun-declaração
var-declaração → tipo-especificador ID ; | tipo-especificador ID [ NUM ] ;
tipo-especificador → int | void
fun-declaração → tipo-especificador ID ( params ) composto-decl
params → param-lista | void
param-lista → param-lista, param | param
param → tipo-especificador ID | tipo-especificador ID [ ]
composto-decl → { local-declarações statement-lista } | { local-declarações } | { statement-lista } | {
}
local-declarações → local-declarações var-declaração | var-declaração
statement-lista → statement-lista statement | statement
statement → expressão-decl | composto-decl | seleção-decl | iteração-decl | retorno-decl
expressão-decl → expressão ; | ;
seleção-decl → if ( expressão ) statement | if ( expressão ) statement else statement
iteração-decl → while ( expressão ) statement
retorno-decl → return ; | return expressão;
expressão → var = expressão | simples-expressão
var → ID | ID [ expressão ]
simples-expressão → soma-expressão relacional soma-expressão | soma-expressão
relacional → <= | < | > | >= | == | !=
soma-expressão → soma-expressão soma termo | termo
soma → + | -
termo → termo mult fator | fator
mult → * | /
fator → ( expressão ) | var | ativação | NUM
ativação → ID ( arg-lista ) | ID ( )
arg-lista → arg-lista, expressão | expressão

```

Fonte: Aulas da Disciplina Compiladores[5]

Como exemplo, segue um código em C- a ser compilado para *assembly*, visto no Algoritmo 4.1.

Algoritmo 4.1 – Algoritmo Exemplo:C-

```

1  int multiplica( int a, int b ){
2      return a*b;
3  }
4
5  void main(void){
6      int a; int b; int res;
7      a = input();
8      b = input();
9      res = multiplica( a, b );
10     output( res );
11 }

```

Seu resultado em *assembly*, visto no [Algoritmo 4.2](#).

Algoritmo 4.2 – Algoritmo Exemplo: *Assembly*

```

1      0: NOP
2      1: jump    10
3      .multiplica
4      2: lw      $1 , 0 ($0 )
5      3: move    $4 , $1
6      4: lw      $1 , 1 ($0 )
7      5: move    $2 , $1
8      6: move    $3 , $4
9      7: mult    $1 , $3 , $2
10     8: move    $29, $1
11     9: jr      $31
12     .main
13     10: input   $1
14     11: sw      $1 , 2 ($0 )
15     12: input   $1
16     13: sw      $1 , 3 ($0 )
17     14: lw      $1 , 2 ($0 )
18     15: sw      $1 , 0 ($0 )
19     16: lw      $1 , 3 ($0 )
20     17: sw      $1 , 1 ($0 )
21     18: jal     2
22     19: move    $1 , $29
23     20: sw      $1 , 4 ($0 )
24     21: output  4 ($0 )
25     22: HALT

```

4.1.2 Arquitetura

As mudanças até esta disciplina na arquitetura foram mínimas, apenas mudanças para otimização e em módulos de memória. Para o suporte de recursão no compilador, na memória de dados criou-se um campo a mais de memória denominado como pilha, capaz de armazenar os dados para a recursão de forma separada e da mesma maneira que o *assembly* de MIPS realiza, a partir do topo e alocando mais espaços com decrementos no *stack pointer*, registrador de número 30 nesta arquitetura.

Nos módulos de memória ainda adicionou-se um *template* do *software* Quartus para a compilação mais rápida do processador independente do tamanho de memória escolhido. Este *template* é capaz de sintetizar as memórias criadas como memórias RAM de fato, diminuindo o tempo de compilação de cerca de 12 minutos para uma média de 1 minuto e 30 segundos para qualquer tamanho de memória.

4.2 Mudanças na Arquitetura

Para fins de contextualização, segue a tabela contendo o conjunto de instruções com as novas inclusas, vistas na [Tabela 1](#).

Diversas instruções novas foram inclusas na arquitetura para que todo o sistema funcionasse de forma correta. Essas instruções compreendem o escopo de transferência de dados entre o HD e a memória principal, em ambos os sentidos, assim como entre o HD e a memória de instruções, apenas nesta direção. Como a troca de contexto também teve de ser implementada, uma instrução de troca de contexto também foi inclusa. Para isso instruções de controle sobre o timer de interrupção também foram criadas. Outras instruções para auxílio de troca de contexto também foram desenvolvidas e serão explicadas com mais detalhes adiante.

Com relação à entrada e saída, o *display* LCD foi utilizado em conjunto com uma nova instrução para o controle de quais caracteres devem ser escritos no LCD de acordo com um número escolhido. O número escolhido define qual configuração de caracteres é escolhida para ser mostrada no *display*.

Tabela 1 – Conjunto de Instruções

OPCode	Instrução	Função
000000	Nop	Ciclo Vazio
000001	Halt	Para Processamento
000010	Add	Soma
000011	Addi	Soma Imediato
000100	Sub	Subtração
000101	Subi	Subtração Imediato
000110	Mul	Multiplicação
000111	Div	Divisão
001000	Slt	<i>Set Less Than</i>
001001	Shr	Desloca Direita
001010	Shl	Desloca Esquerda
001011	Lw	Carrega Palavra
001100	Li	Carrega Imediato
001101	Sw	Guarda Palavra

Tabela 1 – Conjunto de Instruções

OPCode	Instrução	Função
001110	Jump	Salto para Endereço
001111	Jr	Salto para Registrador
010000	Xor	Xor Bit a Bit
010001	And	And Bit a Bit
010010	Or	Or Bit a Bit
010011	Not	Not Bit a Bit
010100	Xori	Xor Imediato
010101	Andi	And Imediato
010110	Ori	Or Imediato
010111	Beq	Branch se Igual
011000	Bneq	Branch se Diferente
011001	Beqz	Branch se Igual a Zero
011010	Bnez	Branch se Diferente de Zero
011011	Bgt	Branch se Maior
011100	Blr	Branch se Menor
011101	Bgtz	Branch se Maior que Zero
011110	Bltz	Branch se Menor que Zero
011111	In	Entrada
100000	Out	Saída
100001	Jal	<i>Jump and Link</i>
100010	Hdinst	Cópia HD para Mem. Instr.
100011	Srprc	Troca Região de Leitura
100100	Src	Troca BIOS para Mem. Instr.
100110	Smenu	Troca Menu LCD
100111	Memhd	Cópia Memória para HD
101000	Lpc	Carrega PC do Processo
101001	Spc	Altera Valor do PC
101010	Lint	Carrega Valor de Interrupção
101011	Ctx	Troca Contexto
101100	Ltb	Carrega Valor Buffer de Transf.
101101	Stb	Salva Valor Buffer de Transf.
101110	Hdmem	Cópia HD para memória
101111	Stm	Set Timer
110000	Rtm	Reset Timer
110001	Lwc	Carrega Reg. de Processo
110010	Swc	Salva Reg. de Processo

Fonte: Autoria Própria

Com o conjunto de instruções definido na [Tabela 1](#), pode-se explicar o funcionamento mais detalhado de cada instrução levando em consideração que a memória de instruções, banco de registradores e contador de programa são divididos em 2 regiões(SO (0) ou Processo (1)), e que a memória de dados é dividida em 11 regiões(SO (0) e Processos (1-10)):

- **Hdinst**: instrução para escrever dados vindos do HD na memória de instruções. Trilha e setor providos de 2 registradores e a região da memória de instruções provida por um valor inteiro sendo 0 ou 1;
- **Srprc**: instrução para mudar a região de leitura de dados para um valor desejado entre 0 e 10. Isso inclui mudar a região de leitura da memória de dados para qualquer valor neste intervalo e os demais módulos para 1 caso o valor seja maior que 0 e 0 caso contrário;
- **Swprc**: mesma funcionalidade que a instrução anterior;
- **Src**: troca a fonte de execução de instruções, sendo esta troca da BIOS para o SO;
- **Smenu**: troca a sequência de caracteres apresentados no LCD com base em um número fornecido. Os menus estão codificados no módulo LCD;
- **Memhd**: instrução para escrever dados vindos do *buffer* de transferência para o HD. Trilha e setor escolhidos por 2 registradores;
- **Lpc**: carrega o valor do PC do processo em execução para o registrador de destino, no caso o registrador \$1;
- **Spc**: altera o valor do PC do processo em execução para o valor contido no registrador lido;
- **Lint**: carrega o valor da interrupção que causou a troca de contexto para o registrador de destino, no caso o registrador \$1;
- **Ctx**: instrução que troca de contexto, neste caso troca a região de leitura da memória de instruções e do PC do processo para o SO e vice-versa, mas não troca a região de leitura nem de escrita do banco de registradores e da memória de dados. Ao executar esta instrução, o valor contido do campo *shamt* da instrução é usado para atualizar o valor de interrupção a ser carregado.
- **Ltb**: carrega o valor do *buffer* de transferência para o registrador de destino;
- **Stb**: altera o valor do buffer de transferência com dados vindos de um registrador;

- **Hdmem**: instrução para escrever dados vindos do HD para o *buffer* de transferência. Trilha e setor escolhidos por 2 registradores;
- **Stm**: atualiza o valor do *timer* de interrupção para que estoure e gere uma interrupção por tempo ao atingir o valor salvo;
- **Rtm**: recomeça a contagem do *timer* a partir do valor 1;
- **Lwc**: carrega o valor do registrador requisitado da região do processo para o endereço de memória requisitado do SO;
- **Swc**: salva o valor lido do endereço de memória requisitado do SO para o registrador da região de processo;

É importante lembrar que o banco de registradores, memória de instruções e PC são divididos em duas regiões, sendo a região número 0 exclusiva do SO e a região de número 1 exclusiva dos processos em execução. Então, quando uma instrução carrega algum valor da região de processos do banco de registradores, esta instrução está lendo os valores dos registradores requisitados da região de número 1.

Assim, pode-se agora definir as inclusões dos módulos em Verilog na arquitetura e uma posterior explicação nas mudanças nos módulos já existentes para que o sistema operacional possa funcionar de forma correta.

4.2.1 BIOS

Construiu-se a BIOS como uma memória ROM capaz de armazenar as instruções necessárias para a partida do sistema. Este módulo então exerce a função de armazenamento e leitura apenas. Para isso, um vetor de tamanho 400 e de dimensão 32bits foi criado, chamado de *mem* que armazena as instruções a serem codificadas. A leitura dos dados é feita utilizando o *clock* da placa FPGA em um *always* visto nas linhas 14 até 17 do [Algoritmo 4.8](#) a partir de um endereço fornecido, sendo este endereço o do PC.

Algoritmo 4.3 – Módulo BIOS em Verilog

```

1
2 module BIOS #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=400)
3 ( address, instOut, clock, autoclock);
4
5     input [(DATA_WIDTH-1):0] address;
6     input clock, autoclock, rst;
7     output reg [(DATA_WIDTH-1):0] instOut;
8     reg [(DATA_WIDTH-1):0] mem [ADDR_WIDTH-1:0];
9

```

```

10     initial
11     begin : INIT
12         /*Instrucoes*/
13     end
14     always @( posedge autoclock )
15     begin
16         instOut <= mem[address];
17     end
18
19 endmodule

```

A leitura de dados de todos os outros módulos de memória como este são feitas na borda de subida do *clock* da placa FPGA para que o *software* Quartus reconheça este *design* como uma memória ROM ou RAM e sintetize-o como tal na placa, reduzindo o tempo de compilação significativamente, sendo este não interferido pelo tamanho da memória. Isto ocorre porque a memória é sintetizada como *memory bits* pelo Quartus e não como *register*, utilizando a memória interna da placa e tirando a sobrecarga de compilação do processo de *fitting*.

Por fim, para que o Quartus sintetize a memória desta maneira, o tamanho da mesma deve ser definido como *parameter* na criação do módulo, visto na linha 2 do [Algoritmo 4.8](#).

O *clock* da placa FPGA é usado como leitura para simular o uso de um *assign*, já que o *clock* vindo direto da placa é bem mais rápido que o *clock* utilizado pelo resto do sistema. No caso, o *clock* da placa é na faixa de 50Mhz e o *clock* do resto do sistema divide este *clock* para chegar na frequência de 50Khz. O *assign* não é utilizado pois o Quartus não sintetiza memória RAM ou ROM sem leitura com *clock*.

4.2.2 HD

Da mesma forma que a BIOS foi desenvolvida, o HD também utiliza a síntese de memória do Quartus, mas dessa vez memória RAM, já que o HD deve ter a capacidade de ser lido e escrito. Para simular as funcionalidade de um HD real, o bloco *initial* também é utilizado para iniciar valores na memória, simulando o fato de que um HD já teria valores salvos previamente, no caso o SO e alguns programas.

Algoritmo 4.4 – Módulo HD em Verilog

```

1 module HardDisk #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH
    =4400, TRACKS=2)
2 ( clock, autoClock, track, trackPos, writeData, readData, hdFlag,
    rst);
3
4 input clock, autoClock, hdFlag, rst;

```

```

5  input[(DATA_WIDTH-1):0] writeData, track, trackPos;
6  output reg [(DATA_WIDTH-1):0] readData;
7  reg[(DATA_WIDTH-1):0] disk [(ADDR_WIDTH-1):0];
8
9  initial begin
10     /* Instrucoes */
11 end
12
13 always @(posedge clock)
14 begin
15     if (hdFlag)
16     begin
17         disk[track*(ADDR_WIDTH/TRACKS) + trackPos] <= writeData;
18     end
19 end
20
21 always@(posedge autoClock)
22 begin
23     readData <= disk[track*(ADDR_WIDTH/TRACKS) + trackPos];
24 end
25
26 endmodule

```

Pode-se ver na linha 1 do [Algoritmo 4.4](#) que o tamanho da do HD é 4400 e que este possui duas trilhas ($TRACKS = 2$). Isto define que cada trilha possui 2200 posições e que cada região pode ser acessada pelo cálculo aritmético $track * (\frac{ADDR_WIDTH}{TRACKS}) + trackPos$.

Este cálculo é necessário pelo seguinte motivo: como a memória não pode ser dividida fisicamente em regiões separadas, esta é criada continuamente com um valor grande e divisível sem restos, assim as regiões podem ser simuladas separadamente através do endereçamento de dados desta maneira. A entrada *track* define qual região a ser acessada, os parâmetros *ADDR_WIDTH* e *TRACKS* definem o tamanho total da memória e a quantidade de regiões respectivamente e a entrada *trackPos* define o endereço a ser acessado na região determinada.

É fácil ver como esta divisão por endereçamento é realizada com um exemplo. Suponha que um dado deve ser escrito na região número 1 na posição 15, então o cálculo $track * (\frac{ADDR_WIDTH}{TRACKS}) + trackPos$ resulta no valor 2215, endereço correspondente à região número 1 na posição 15. O mesmo vale para uma escrita na região número 0 na posição 38, onde o dado seria escrito no endereço 38.

Assim, os dados nunca se interferem e se sobrescrevem, resultando numa divisão de regiões distintas da mesma maneira se estivessem em regiões fisicamente separadas.

4.2.3 Buffer Bank

Este módulo possui a finalidade de armazenar o valor da região de escrita e leitura atual assim como o valor a ser escrito para transferência. Através das instruções *Srprc* e *Swprc* vistas na Tabela 1, pode-se mudar o valor das regiões de escrita e leitura neste módulo armazenadas nos registradores *ReadProcessId* e *WriteProcessId*. Como explicado anteriormente, os valores de leitura e escrita da memória de instrução e do banco de registradores variam apenas entre 0 e 1, pois existem apenas 2 regiões para estes módulos, então os valores de saída *rwriteId* e *rpId* são destinados à estes módulos, já que para valores de leitura e escrita acima de 0 a saída é 1 e 0 caso contrário.

Algoritmo 4.5 – Módulo *Buffer Bank* em Verilog

```

1 module BufferBank #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH
  =32)
2 (WriteData, clock, readpId, writepId, rpId, rwritepId, setProcess
  , TransfBuffer, bufferData, bw);
3
4 input signed [(DATA_WIDTH-1):0] WriteData, bufferData;
5 input clock, bw;
6 input [1:0] setProcess;
7 output [(DATA_WIDTH-1):0] readpId, writepId, rpId, rwritepId;
8
9 reg [(DATA_WIDTH-1):0] ReadProcessId;
10 reg [(DATA_WIDTH-1):0] WriteProcessId;
11 output reg [(DATA_WIDTH-1):0] TransfBuffer;
12
13 initial
14 begin: INIT
15     ReadProcessId <= 0;
16     WriteProcessId <= 0;
17     TransfBuffer <= 0;
18 end
19
20 always @(posedge clock)
21 begin
22     if( bw )
23         TransfBuffer <= bufferData;
24     else if( setProcess == 2'b01 )
25         ReadProcessId <= WriteData;
26     else if( setProcess == 2'b10 )
27         WriteProcessId <= WriteData;
28     else

```

```

29     begin
30     end
31 end
32
33 assign readpId = ReadProcessId;
34 assign writepId = WriteProcessId;
35 assign rpId = ReadProcessId > 0 ? 1 : 0;
36 assign rwritepId = WriteProcessId > 0 ? 1 : 0;
37
38 endmodule

```

O registrador *TransferBuffer* possui dois propósitos: transferência de dados entre HD e memória e transferência de dados entre processo e SO.

Para a transferência de dados entre HD e memória, as instruções *Hdmem* e *Memhd* são utilizadas para que dados entrem no *buffer* e saiam deste respectivamente. Assim, para carregar um valor do HD para a memória, utiliza-se a instrução *Hdmem* para que os dados sejam transferidos ao *TransferBuffer* e a instrução *Ltb* para que os dados do *buffer* sejam carregados para a memória. O mesmo vale para salvar dados da memória no HD, só que no sentido contrário, utilizando a instrução *Stb* para salvar um dado no *buffer* e depois a instrução *Memhd* para salvar este dado do *buffer* no HD.

Além disto, o registrador *TransferBuffer* tem seu propósito para a entrada e saída de um processo em execução. Como este processo não pode acessar os dispositivos de E/S por conta própria e somente pelo intermédio do SO, o processo em questão precisa transferir dados com o SO de alguma maneira. Assim, para realizar uma saída de dados, o processo salva o valor ser mostrado no *buffer* e o SO carrega este valor para realizar o *output*. Para realizar uma entrada de dados o SO salva o valor de entrada no *buffer* e o processo carrega-o para sua memória.

4.2.4 *Interruption Controller*

O módulo controlador de interrupção é responsável por trocar a região de leitura da memória de instruções ao receber uma interrupção por tempo (estouro do *quantum*) ou ao executar-se uma instrução de troca de contexto. Pode-se ver isto na [Algoritmo 4.6](#).

Algoritmo 4.6 – Módulo *Interruption Controller* em Verilog

```

1 module InterruptionController(timerInt, context, changeContext,
   signalOut, region, instructionRegion, readpId, clock);
2
3     input timerInt, changeContext, clock;
4     input [2:0] context;
5     input [31:0] readpId;

```

```
6  output reg[2:0] signalOut;
7  output reg[3:0] instructionRegion;
8  output reg region;
9  wire [3:0] selectRegion;
10
11 assign selectRegion = instructionRegion != 0 ? 0 : readpId ;
12
13 initial begin
14     region <= 0;
15     instructionRegion <= 0;
16 end
17
18 always @( posedge clock )
19 begin
20     if( timerInt || changeContext )
21     begin
22         region <= !region;
23         instructionRegion <= selectRegion;
24     end
25 end
26
27 always @( posedge clock )
28 begin
29     if( changeContext )
30         signalOut <= context;
31     else if( timerInt )
32         signalOut <= 3'b000;
33 end
34
35 endmodule
```

A saída nomeada de *region* vista na linha 22 do [Algoritmo 4.6](#) define como a troca de contexto é realizada. Quando um sinal de estouro de *timer* ou troca de contexto, a região de leitura da memória de instruções troca de 0 para 1 ou de 1 para 0. Isto ocorre porque a região 0 corresponde ao SO e a 1 aos demais processos. O estouro de *timer* só ocorre quando um processo qualquer está executando, e como o processo não tem controle sobre o *timer*, o sinal troca automaticamente a região de execução sem consentimento do processo para que o SO possa realizar a troca de contexto.

Já a troca de contexto por intermédio de instrução pode ser realizada pelo SO ou por qualquer processo. Quando o SO necessita executar um processo, basta executar a instrução de troca de contexto passando o controle para o processo. O mesmo ocorre quando

um processo precisa realizar E/S ou sinalizar término, passando o controle arbitrariamente ao SO.

Isto leva em conta outro processo importante na troca de contexto, o valor da interrupção. O valor da interrupção é utilizado pelo SO para saber qual tipo de interrupção ocorreu e o que fazer após este evento. Este valor é alterado pelo campo *shamt* da instrução de troca de contexto, sendo este valor escolhido pelo processo que executa a instrução. Sendo assim, um processo que necessita realizar uma entrada de dados executa esta instrução com o valor que indica a entrada de dados para que o SO faça-o corretamente. Por fim, uma interrupção de tempo automaticamente altera o valor de interrupção para 0 para sinalizar o SO, já que o processo interrompido sofre preempção contra sua vontade.

4.2.5 *Timer* de Interrupção

O módulo de *timer* é responsável por gerar as interrupções de *quantum* ao se atingir uma certa contagem. O principal objetivo é gerar estas interrupções sem a interferência de nenhum processo, automaticamente, como visto no [Algoritmo 4.7](#).

Algoritmo 4.7 – Módulo *Timer* de Interrupção em Verilog

```

1  module Timer( clock, reset, timerReset, interval, timerWrite,
    timerBlock, ReadId, halt, interrupt);
2
3  input clock, reset, timerWrite, timerReset, timerBlock, halt;
4  input[31:0] interval, ReadId;
5  output reg interrupt;
6  reg[31:0] counter;
7  reg[31:0] savedInterval;
8  wire block;
9
10 initial begin
11     savedInterval <= 30;
12     counter <= 1;
13 end
14
15 assign block = (ReadId == 0 || halt || timerBlock) ? 1 : 0;
16
17 always @(posedge clock)
18 begin
19     if( reset || timerReset )
20     begin
21         counter <= 1;
22         interrupt <= 1'b0;

```



```

23     end
24     else if( timerWrite )
25     begin
26         savedInterval <= interval;
27         counter <= 1;
28         interrupt <= 1'b0;
29     end
30     else if( counter == savedInterval )
31     begin
32         counter <= 1;
33         interrupt <= 1'b1;
34     end
35     else
36     begin
37         if( !block )
38             counter <= counter + 1;
39             interrupt <= 1'b0;
40         end
41     end
42
43 endmodule

```

O *timer* gera a interrupção quando o registrador *counter* atinge o valor salvo no registrador *savedInterval*. Este último registrador pode ter seu valor alterado por meio da instrução *Stm* vista na [Tabela 1](#).

Percebe-se na linha 12 do [Algoritmo 4.7](#) que o contador começa em 1 e mesmo quando recomeça a contagem este começa em 1. Isto deve-se ao fato de que o usuário do SO pode preferir que os programas executem sem preempção, sendo assim o valor do registrador *savedInterval* pode ser alterado para 0, assim a contagem nunca estoura o limite nunca gerando a interrupção, já que a contagem começa em 1 e só estoura ao chegar 0.

Assim como citado anteriormente, é possível recomeçar a contagem a partir da instrução *Rtm* e alterar o valor de estouro com *Stm*. Por conta de que algumas sequências de instruções não podem ser interrompidas durante a execução, como por exemplo salvar algum valor no *buffer* de transferência para depois chamar o SO para a saída dos dados, e que o *timer* não pode estourar durante a execução do SO, a variável *block*, vista na linha 15 do [Algoritmo 4.7](#) determinar se o contador pode prosseguir a contagem ou não. Caso a região de leitura esteja em 0 (SO executando) ou o processador esteja em *halt* ou alguma instrução que não pode ser interrompida (delimitada pela entrada *timerBlock*), o contador não pode prosseguir para que a interrupção por tempo não ocorra.

4.2.6 Saída de Dados

A saída de dados do sistema era previamente realizada a partir dos *displays* de 7 segmentos apenas, com 3 *displays* para os dados de saída e 2 *displays* para o valor do PC atual. Como este projeto trata de execuções mais complexas, o *display* LCD foi incorporado na saída de dados.

Para melhor visualização dos resultados, 4 *displays* de 7 segmentos são utilizados para o valor do PC e 2 *displays* para mostrar o valor do processo em execução. No *display* LCD foram utilizados vários *menus* para cada caso do SO que são capazes de mostraram os dados de entrada em tempo real e os dados de saída com o uso da instrução *Out*.

Os *menus* são selecionados a partir da instrução *Smenu*. Esta instrução serve apenas para modificar o valor de um registrador no módulo do *display* LCD, sendo este registrador responsável por selecionar qual menu está sendo mostrado no momento. Ao modificar este registrador para um valor desejado, o determinado *menu* é mostrado na tela. Como não foram criadas outras instruções para o controle do *display* LCD, todos os *menus* são *hardcoded* e caso deseje-se criar um novo menu, deve-se adicionar seu código no módulo LCD e seu respectivo valor para acessá-lo.

Os *menus* possuem a finalidade de auxiliar nas mensagens de comunicação do SO com o usuário, mostrando a criação de arquivos, renomeação e deleção, além da escolha de ações do SO e da escolha de quais processos serão executados e em qual ordem.

4.3 BIOS em C-

O código da BIOS em C- é bem simples e serve seu propósito de teste de partida e cópia do SO para a memória. A implementação do mesmo pode ser vista no [Algoritmo 4.8](#).

Algoritmo 4.8 – BIOS em C-

```
1 void main( void ){
2     int i;
3     showPrompt( 0 );
4
5     i = input();
6
7     output(i);
8
9     i = i * 3;
10
11     output(i);
12 }
```

```
13     i = 0;
14     while( i < 1880 ){
15         cpyHDTToInstr( 0, 0, i );
16         i = i + 1;
17     }
18     output( i );
19 }
```

As únicas funções observadas aqui são testar as chaves de entrada de dados da FPGA através do *input* de dados, testar a transferência de dados, a aritmética e o banco de registradores com a multiplicação do valor entrado por 3 e testar a saída de dados no *display* LCD mostrando o resultado da multiplicação. Por fim, o SO é copiado do HD para a memória de instruções para que seja executado após o término da BIOS. No código em C- não é possível ver a troca para o SO, mas em seu código *assembly* a última instrução trata-se da troca do fluxo da BIOS para o SO.

4.4 Lógica e Implementação do SO em C-

O sistema operacional em questão não passa de um programa desenvolvido em C- que executa no processador desenvolvido anteriormente. Mesmo sendo apenas um programa em execução, o SO possui privilégios diferenciados em comparação com os outros programas carregados em memória, como a habilidade de manipular dados no HD, controlar a execução de outros programas, determinar segmentos de memória para cada programa e até controlar as operações de E/S.

Como até o momento foram definidos todos os módulos implementados em Verilog para a arquitetura e todas as instruções desenvolvidas para dar suporte ao SO e à troca de contexto, é possível explicar e definir as funcionalidades do SO implementadas em C-. O código completo pode ser visto no [Apêndice A, Algoritmo A.1](#).

4.4.1 Funções do Sistema

Como parte importante do SO, existem as funções de sistema para o auxílio da execução. Estas funções não passam de chamadas que são interpretadas pelo compilador e transformada no código em *assembly* correspondente, ou seja, são funções que existem apenas em *assembly* e podem ser chamadas pelo programa através de sua chamada de função. Um exemplo é a função de *Output*, uma função que não possui uma definição anterior no código mas pode ser chamada livremente sem erro de compilação, sendo esta função interpretada pelo compilador e transformada no conjunto de códigos em *assembly* necessários para que funcione como esperada.

Por conseguinte, diversas outras funções de sistema foram criadas para que o SO possa realizar operações em mais baixo nível, chamando rotinas pré definidas em *assembly*. Claramente os códigos em *assembly* não são vistos no código do SO e muito menos pelo usuário e sim pelo projetista do compilador que disponibiliza uma espécie de API do processador através destas funções.

Abaixo encontram-se as funções disponíveis para o uso do SO que invocam essas rotinas e o que cada uma desempenha:

- **int input():** realiza entrada de dados no sistema;
- **void output(int x):** realiza saída de dados da variável x;
- **void showPrompt(int id):** muda o *menu* LCD para o id desejado;
- **void setWriteProcess(int x):** altera a região de escrita para o valor de x;
- **void setReadProcess(int x):** altera a região de leitura para o valor de x;
- **void toggleProcess(int x):** troca de contexto e muda o valor de interrupção para o valor de x;
- **int loadInterrupction():** carrega o valor de interrupção em uma variável;
- **int loadPC():** carrega o valor do contador de programa do processo que estava em execução;
- **void savePC(int x):** altera o valor do contador de programa do processo em execução para o valor de x;
- **void setTimer(int x):** altera o valor de estouro do *timer* de interrupção para o valor de x, ou seja, atualiza o valor do *quantum*;
- **void resetTimer():** recomeça a contagem do *timer* de interrupção;
- **int loadBuffer():** carrega o valor contido no *buffer* de transferência para uma variável;
- **void storeBuffer(int x):** salva o valor da variável x no *buffer* de transferência;
- **void cpyHDTtoInstr(int region, int track, int pos):** copia o dado contido na trilha indicada por *track* e na posição indicada por *pos* do HD para a região da memória de instruções indicada por *region*;
- **void cpyMemToHD(int track, int pos):** copia o valor contido no *buffer* de transferência para a trilha *track* e posição *pos* do HD;

- **void shiftBits(int a, int b)**: concatena dois dados de entrada *a* e *b* para formar um dado de 32bits. Usado na criação de instruções por meio de *input*, já que os dados que entram no sistema são limitados a 16bits pelas chaves da placa FPGA;
- **void loadRegs(int x)**: carrega os registradores salvos na posição *x* de volta no banco de registradores de processos (região 1 do banco);
- **void saveRegs(int x)**: salva o banco de registradores de processos na região de memória *x*. Esta região de memória faz parte do domínio do SO e somente ele pode acessá-la;

4.4.2 Inicialização do Sistema

Antes de todas as operações principais do SO estarem prontas para o uso, é necessário que o sistema realize o procedimento de inicialização para que todas as variáveis e vetores estejam iniciados com os valores corretos para o uso.

Primeiramente, as variáveis presentes no [Algoritmo 4.9](#) são parte do [Algoritmo A.1](#) e compõem o sistema de arquivos do sistema operacional. Estas variáveis são então inicializadas com os valores necessários para que o SO seja capaz de carregar os arquivos já presentes no HD do sistema. O que cada variável representa será discutido nas próximas seções.

Algoritmo 4.9 – Variáveis de Controle de Arquivos

```

1  int  fileLocation [35];
2  int  fileStart [35];
3  int  fileEnd [35];
4  int  fileId [35];
5  int  fileExists [35];
6  int  fileSize [35];
7  int  fileCounter;
8  int  trackCounter;
9  int  filesStart;
```

Por fim, existem constantes usadas para o controle dos *menus* no *display* LCD, e estas variáveis também podem ser encontradas no começo do código do SO, vistas no *snip* do [Algoritmo 4.10](#). Todas exceto *MAXSECTOR* e *MAXTRACKS* são utilizadas para seleção de *menus* e recebem valores na ordem crescente começando de 0. Assim, quando precisa-se trocar de *menu*, basta chamar a função respectiva com a constante desejada.

Algoritmo 4.10 – Variáveis de Controle dos Menus LCD

```

1  int  OSSTART;
```

```
2  int FWRITEA;
3  int FWRITEB;
4  int FWRITEC;
5  int FRENAME;
6  int FDELETE;
7  int BATCHJOB;
8  int PROCESSRUN;
9  int PSELECT;
10 int IPROCESS;
11 int NPROCESS;
12 int FCONFIRM;
13 int FNFND;
14 int FAEXT;
15 int TIMER;
16
17 int MAXSECTOR;
18 int MAXTRACKS;
```

As variáveis *MAXSECTOR* e *MAXTRACKS* são utilizadas no controle de arquivos e recebem, respectivamente, o tamanho máximo de cada trilha do HD (2200 no caso) e a quantidade de trilhas (2 no caso).

4.4.3 *Prompt* de Comando

Uma parte de grande importância do sistema operacional é o *prompt* de comando que faz interface com o usuário para que este possa escolher o que fazer durante a execução do SO. A apresentação do *prompt* começa após a inicialização do sistema visto na função *main* do *snip* presente no [Algoritmo 4.11](#).

Algoritmo 4.11 – *Prompt* de Comando

```
1  void main(void){
2      int i;
3      int file;
4      i = 10;
5
6      showPrompt( OSSTART );
7      startSystem();
8      while( i != 0 ){
9          showPrompt( OSSTART );
10         i = input();
```

```
11     if( i == 1 ){
12         writeFile();
13     }
14     else if( i == 2 ){
15         renameFile();
16     }
17     else if( i == 3 ){
18         deleteFile();
19     }
20     else if( i == 4 ){
21         roundRobin();
22     }
23 }
24 }
```

Após a execução da função *startSystem*, o SO entra em um laço controlado pela variável *i* que por sua vez é definida pelo usuário. Dependendo da entrada do usuário, uma rotina específica do SO é executada. Caso o usuário decida terminar a execução do SO, basta escolher o número 0 dando fim à execução.

Para maior interatividade com o usuário, o *prompt* disponibiliza um *menu* LCD definido pela constante *OSSTART* onde o usuário pode ver quais opções escolher. Estas opções compreendem: 1 para escrever uma arquivo, 2 para renomear um arquivo, 3 para deletar um arquivo e 4 para executar programas.

4.4.4 Gerenciamento de Arquivos

O sistema de arquivos é gerenciado pelo SO através das variáveis descritas na seção que trata da inicialização do sistema, sendo estas vistas no [Algoritmo 4.9](#). Estas variáveis possibilitam que o SO encontre os arquivos presentes no HD, possa carregá-los, modifica-los e até criar novos arquivos.

Primeiramente, os vetores contidos na linha 1 até a linha 6 do [Algoritmo 4.9](#) são responsáveis por guardarem todas as informações sobre os arquivos. O vetor *fileLocation* guarda em qual trilha o determinado arquivo está. O vetor *fileStart* determina em qual posição desta trilha o arquivo começa. O vetor *fileEnd* armazena em qual posição do HD o arquivo termina, lembrando que o arquivo pode começar em uma trilha e acabar em outra. O vetor *fileId* guarda qual o *ID* do arquivo, representado por um número inteiro de 32bits. O vetor *fileExists* mantém o controle sobre quais arquivos existem ou não, para que o SO saiba que está procurando arquivos ainda ativos e não deletados ou inexistentes. Por fim, o vetor *fileSize* determina a quantidade de linhas de um arquivo, ou seja, o quanto o

SO deve ler do HD para compreender todo o arquivo.

As variáveis presentes na linha 7 a 9 do [Algoritmo 4.9](#) são então usadas para controlar o total dos arquivos. A variável *fileCounter* mantém a contagem de quantos arquivos existem e serve para o SO saber onde parar de procurar arquivos ou até onde começar a escrever um arquivo nos vetores de arquivos. A variável *trackCounter* define em qual trilha o último arquivo se localiza para que na hora de escrita de um novo arquivo, este possa começar a ser escrito na trilha correta. Por fim a variável *filesStart* define onde o primeiro arquivo de sistema começa, já que o primeiro arquivo do HD é o SO localizado na trilha 0 posição 0.

Assim, pode-se ver que para localizar um arquivo, basta saber seu *ID* e procurá-lo nos vetores de arquivo. Localizada a posição deste *ID* nos vetores, basta ver onde o arquivo começa, onde terminar, qual trilha se localiza e qual seu tamanho e, com estas informações em mãos, basta carregar o arquivo ou até deletá-lo.

4.4.4.1 Carregamento de Arquivos/Programas

Para que o SO possa carregar um programa na memória, ou seja, ler um arquivo do HD e copiar seus dados para a região 1 da memória de instruções, este primeiro precisa localizar o arquivo nos vetores de arquivos. Para isso, executa-se a rotina chamada de *loadCode()* definida no código do SO do [Algoritmo A.1](#) e vista no *snip* do [Algoritmo 4.12](#).

Algoritmo 4.12 – Carregando um Programa

```
1 void loadCode( int id ){
2     int i; int it; int t; int s;
3
4     i = searchFile( id );
5
6     if( i != 0 ){
7         s = 0;
8         it = fileStart[i];
9         t = fileLocation[i];
10        nop();
11
12        while( s < fileSize[i] ){
13            cpyHDTtoInstr( 1, t, it );
14            if( it == MAXSECTOR ){
15                it = 0;
16                t = t + 1;
17            }
18            else{
```



```

19         it = it + 1;
20     }
21     s = s + 1;
22 }
23 }
24 }
```

Neste procedimento, o arquivo com o *id* definido na hora da chamada de função deve ser carregado na memória de instruções. Para este caso, a posição deste *id* é procurada no vetor *fileId* de forma sequencial a partir da primeira posição. Ao encontrar sua posição no vetor, o processo de leitura do HD e passagem de dados para a memória de instruções pode começar.

Salvo na variável *i* está a posição do arquivo nos vetores, então para saber em qual trilha o arquivo começa e em qual posição desta trilha, basta carregar os valores dos vetores *fileLocation* e *fileStart* na posição *i*, respectivamente. Agora que sabe-se que o arquivo localiza-se na trilha indicada pela variável *t* e começa na posição indicada pela variável *it*, basta começar a ler o arquivo iterando sobre *it* até que o tamanho máximo do arquivo seja atingido.

Esta rotina pode ser vista no *while* da linha 12 do [Algoritmo 4.12](#), onde o arquivo é lido até que o tamanho todo deste seja sanado. Assim, invoca-se a rotina *cpyHDTToInstr()* descrita anteriormente para carregar os dados da trilha *t* e posição *it* do HD para a região de número 1 da memória de instruções onde se localizam os programas de usuário. Além disso, nas linhas 14 até 19 encontra-se a verificação de quando chega-se no final da trilha e deve-se continuar a ler o arquivo na próxima trilha a partir da primeira posição desta.

Um fato a ser esclarecido é a presença da chamada *nop()* antes do começo do laço de cópia. Como todo arquivo deve ser carregado no começo da memória de instruções até seu término, ao se carregar um novo arquivo, a contagem de endereços onde copiar o programa na memória de instruções deve recomeçar em 0 e a instrução *NOP* foi utilizada para *resetar* esta contagem em *hardware* para que os arquivos novos sempre sejam escritos a partir do começo da memória.

4.4.4.2 Escrita de Arquivos

Para a escrita de programas, basta saber onde começar a escrever, ou seja, basta utilizar os valores das variáveis *fileCounter* e *trackCounter* para saber onde começar a escrever. A rotina do SO para escrita de arquivos pode ser vista no *snip* do [Algoritmo 4.13](#).

Algoritmo 4.13 – Rotina de Escrita de Arquivos

```

1 void writeFile(void){
2     int i; int instruction; int id; int aux;
```

```
3  int sa; int sb; int control;
4  int wLoc; int track;
5
6  control = 0;
7  sa = 0;
8  sb = 0;
9  wLoc = fileEnd[fileCounter-1];
10 track = trackCounter;
11
12 control = 0;
13 showPrompt( FWRITEA );
14 id = input();
15 aux = searchFile(id);
16 if( aux != 0 ){
17     showPrompt(FAEXT);
18     output(id);
19     return;
20 }
21 while( control != (0-1)){
22     showPrompt( FWRITEB );
23     sa = input();
24     sb = input();
25     instruction = shiftBits( sa, sb );
26     if( wLoc == MAXSECTOR ){
27         wLoc = 0;
28         track = track + 1;
29     }
30     else{
31         wLoc = wLoc + 1;
32     }
33     storeBuffer( instruction );
34     cpyMemToHD( track, wLoc );
35     i = i + 1;
36     output( i );
37     showPrompt( FWRITEC );
38     control = input();
39 }
40 if( i != 0 ){
41     showPrompt(FCONFIRM);
```

```
42     control = input();
43     if( control == 1 ){
44         fileLocation[fileCounter] = trackCounter;
45         fileStart[fileCounter] = fileEnd[fileCounter-1]+1;
46         fileEnd[fileCounter] = wLoc-1;
47         fileId[fileCounter] = id;
48         fileExists[fileCounter] = 1;
49         fileSize[fileCounter] = i;
50         fileCounter = fileCounter + 1;
51         trackCounter = track;
52     }
53 }
54 }
```

A rotina de escrita precisa saber onde começar a escrever o novo arquivo, para isto utiliza-se da variável *trackCounter* para saber em qual trilha começar a escrever e, como os arquivos são escritos sequencialmente, basta saber onde o último arquivo terminar e começar escrever uma posição depois, como visto na linha 9 do [Algoritmo 4.13](#).

Antes da escrita, verifica-se se o arquivo já existe, se sim mostra-se uma mensagem de erro no LCD com o *id* do arquivo e termina-se a rotina. Caso contrário o laço de escrita é executado. Neste laço o mesmo ocorre para a leitura de programas do HD, onde começa-se em uma posição de uma trilha e escreve-se sequencialmente até que o limite da trilha seja atingido, passando-se para a próxima trilha na primeira posição.

O laço de escrita é controlado pela variável *control* que define quando o arquivo deve ser parado de ser escrito, ou seja, quando o usuário entra com -1. Para escrever uma linha do arquivo precisa-se de um dado de 32bits e como a entrada de dados é limitada por 16bits de chaves da FPGA, utiliza-se a rotina *shiftBits()* capaz de concatenar dois dados de 16bits formando um de 32bits. Esta rotina realizar um *shift* de 16 para a direita do segundo dado fornecido (*sb*) e depois executa um *OR bit* a *bit* entre *sa* e *sb* deslocado. Isto resulta num dado de 32bits, que no caso pode formar uma instrução.

Por fim, na linha 40 verifica-se se alguma linha foi escrita e se sim, caso o usuário queira escrever o arquivo ao fornecer a entrada 1 para a verificação na linha 43. Com isso, o arquivo é escrito no sistema de arquivos ao se colocar todos os dados correspondentes à sua posição, tamanho, início, fim, e *id* nos vetores correspondentes na posição após o último arquivo escrito. Depois disto, o contador de arquivos *fileCounter* é incrementado e a variável de controle de trilhas é atualizada para o valor da trilha onde o último dado do arquivo foi escrito.

Com isso, tem-se o arquivo escrito no HD e inserido nos vetores de arquivos, sendo

este agora um arquivo que pode ser carregado para a memória pela rotina *loadCode()* descrita anteriormente.

4.4.4.3 Renomeação de Arquivos

O processo de renomeação de arquivos é bem simples se comparado aos demais. Basta analisar o [Algoritmo 4.14](#). Percebe-se que basta apenas encontrar o arquivo a ser renomeado e trocar o valor de seu *id* no vetor *fileId* pelo novo *id* desejado. Somente com este código já é possível realizar esta ação.

Algoritmo 4.14 – Rotina de Renomeação de Arquivos

```
1 void renameFile( void ){
2     int i; int flag;
3     int id; int newId;
4
5     i = 0;
6     flag = 99;
7     showPrompt( FRENAME );
8     id = input();
9     newId = input();
10
11     while( i < fileCounter ){
12         if( fileExists[i] == 0 )
13             i = 99;
14         else if( fileId[i] == id ){
15             flag = i;
16             i = 99;
17         }
18         i = i + 1;
19     }
20     if( flag < fileCounter ){
21         fileId[flag] = newId;
22     }
23     else{
24         output(FNFND);
25         output(id);
26     }
27 }
```

4.4.4.4 Deleção de Arquivos

A rotina para deletar arquivos começa da mesma maneira que a renomeação de arquivos, buscando a posição do arquivo a ser deletado nos vetores. O laço de busca começa na linha do 7 [Algoritmo 4.15](#).

Algoritmo 4.15 – Rotina para Deletar Arquivos

```
1 void deleteFile( void ){
2     int i; int control;
3     showPrompt( FDELETE );
4     FILEID = input();
5     i = filesStart;
6     control = 1;
7     while( control != 0 ){
8         if( fileExists[i] == 0 ){
9             control = 0;
10            showPrompt(FNFND);
11            output(FILEID);
12        }
13        else{
14            if( fileId[i] == FILEID ){
15                fileExists[i] = 0;
16                control = 0;
17                if( fileCounter != i ){
18                    while( fileExists[i+1] != 0 ){
19                        fileLocation[i] = fileLocation[i+1];
20                        fileStart[i] = fileStart[i+1];
21                        fileEnd[i] = fileEnd[i+1];
22                        fileId[i] = fileId[i+1];
23                        fileExists[i] = fileExists[i+1];
24                        fileSize[i] = fileSize[i+1];
25                        i = i + 1;
26                    }
27                    fileCounter = fileCounter - 1;
28                }
29            }
30            i = i + 1;
31        }
32    }
33 }
```

Caso o arquivo não exista, uma mensagem de erro no *display* LCD é gerada nas linhas 10 e 11. Caso contrário o arquivo tem seu vetor *fileExists* atualizado para 0 e todos os arquivos consequentes são deslocados para trás no laço da linha 18. Isto faz com que o arquivo deletado seja apagado da lista de arquivos e que todos os arquivos assumam a ordem correta ao tomarem o lugar do arquivo deletado.

4.4.5 Gerenciamento de Processos e Memória

O gerenciamento de processos e de regiões de memória são feitos diretamente a partir da rotina do [Algoritmo 4.16](#). Nesta rotina o usuário escolhe a quantidade de programas que serão executados, quais serão estes programas, qual a ordem de execução e se haverá preempção.

A rotina a ser executada é o *Round Robin*, uma simples fila circular de processos que itera sobre as escolhas do usuário uma a uma durante o escalonamento.

Algoritmo 4.16 – Rotina de Escalonamento de Programas

```
1 void roundRobin( void ){
2     int it; int i; int n; int switch; int k; int end; int ant;
3     int p;
4
5     it = 0;
6     switch = 1;
7     end = 0;
8
9     startVets();
10    showPrompt(TIMER);
11    i = input();
12    setTimer( i );
13    n = nProcess;
14    savePC(0);
15    while( n > 0 ){
16        if( switch != 0 ){
17            if( end == 1 ){
18                if( it < n-1 ){
19                    k = it;
20                    while( k < n-1 ){
21                        roundPC[k] = roundPC[k+1];
22                        roundQueue[k] = roundQueue[k+1];
23                        roundStatus[k] = roundStatus[k+1];
24                        k = k + 1;
```

```
24         }
25     }
26     if( it == n - 1 )
27         it = 0;
28     n = n - 1;
29 }
30 else{
31     if( it == 0 )
32         ant = n-1;
33     else
34         ant = it - 1;
35     roundPC[ant] = loadPC();
36     storeRegisters( roundStatus[ant]-1 );
37 }
38 loadRegisters( roundStatus[it] -1 );
39 loadCode( roundQueue[it] );
40 savePC( roundPC[it] );
41     output( roundPC[it] );
42 }
43
44 p = roundStatus[it];
45
46 if( INTERRUPTFLAG == 0 )
47     resetTimer();
48 if( INTERRUPTFLAG == 3 )
49     resetTimer();
50
51 setWriteProcess(p);
52 setReadProcess(p);
53 toggleProcess(0);
54 setWriteProcess(0);
55 setReadProcess(0);
56 INTERRUPTFLAG = loadInterruption();
57
58 k = roundQueue[it];
59
60 showPrompt( IPROCESS );
61 output( k );
62 showPrompt( PROCESSRUN );
```

```

63
64     /* Timer */
65     if( INTERRUPTFLAG == 0 ){
66         end = 0;
67         switch = 1;
68         if( it == n - 1 )
69             it = 0;
70         else
71             it = it + 1;
72     }
73     /* Input */
74     else if( INTERRUPTFLAG == 1 ){
75         end = 0;
76         switch = 0;
77         systemInput();
78     }
79     /* Output */
80     else if( INTERRUPTFLAG == 2 ){
81         end = 0;
82         switch = 0;
83         systemOutput();
84     }
85     /* End */
86     else if( INTERRUPTFLAG == 3 ){
87         end = 1;
88         switch = 1;
89         if( n == 1 )
90             n = 0;
91     }
92 }
93 }

```

Antes dos detalhes da escolha dos processos, vale explicar que o vetor *roundQueue* guarda os *ids*, *roundStatus* guarda qual a região de memória do processo e *roundPC* guarda qual o valor do contador de programa do processo. Com estes 3 vetores pode-se manter os dados de todos os processos em execução durante o procedimento.

Primeiramente, a rotina *startVets()* inicia para que o usuário selecione os programas a serem executados. Dentro desta rotina o usuário deve informar a quantidade de processos a serem executados e logo depois o *id* dos processos a serem executados. Caso o *id* não

exista uma mensagem de erro no *display* LCD é mostrada. Por fim, o valor do *id* é colocado na posição atual do iterador no vetor *roundQueue* e a região de memória onde o processo deve executar é salva no vetor *roundStatus*. As regiões de memória começam em 1 para o primeiro processo e aumentam iterativamente para cada processo, já que a memória de dados possui 11 regiões, 0 para o SO e de 1 a 10 para os processos. Por fim, para todos os processos, o valor do contador de programa é iniciado em 0 já que nenhum processo está em execução.

Após a inicialização dos vetores e a escolha de processos pelo o usuário, o próximo passo é escolher se os processos serão executados com ou sem preempção. Esta etapa ocorre nas linhas 10 e 11 no [Algoritmo 4.16](#), onde o usuário deve entrar com um valor para o *quantum* que deverá ser cedido a cada processo. Caso o usuário escolha o valor 0, os processos serão executados sem preempção e qualquer valor maior gera preempção da quantidade que o usuário inseriu em ciclos de *clock*, ou seja, um *quantum* de 30 interrompe um processo depois de 30 ciclos de *clock*.

Por fim, dentro do laço da linha 14, os processos serão escalonados até que todos terminem, visto pela variável de controle *n* que indica quantos processos ainda estão executando. Dentro deste laço então ocorrem 3 situações:

- Interrupção por estouro de *quantum*;
- Interrupção por E/S;
- Interrupção por fim de processo;

Cada uma das situações é tratada de uma diferente maneira que será explicada adiante. Antes disto porém, é necessário definir o que as variáveis de controle *end* e *switch* realizam dentro do laço.

A variável *end* é utilizada quando um processo termina ou é interrompido por estouro de *quantum*. Já a variável *switch* é utilizada quando um processo é interrompido por estouro de *quantum* somente.

Primeiramente deve-se definir como o primeiro processo começa a ser executado, ou seja, como caracteriza-se a primeira iteração deste laço. Pelas linhas 5 e 6 do [Algoritmo 4.16](#), observa-se que as variáveis *switch* e *end* são iniciadas com os valores 1 e 0 respectivamente. Isto força a condição em que o processo atual seja carregado do HD para a memória de instruções para ser executado posteriormente. Mesmo que as ações de salvar o contexto dos registradores e o PC sejam executadas para o primeiro processo, isto não interfere na execução pois as memórias estarão zeradas antes de todo começo de execução do escalonador.

Para que o código de um processo seja carregado do HD para a memória de instruções, basta saber o *id* do processo em questão para que a função *loadCode* faça seu trabalho. O funcionamento desta função já foi detalhado anteriormente, mas vale lembrar que ela busca o *id* do programa nos vetores de arquivos e, a partir dos dados sobre localização deste no HD, basta carregar as instruções para a memória de instruções.

A seguir serão explicados como os 3 tipos interrupção são tratadas no escalonamento dos processos.

4.4.5.1 Interrupção por Estouro de *Quantum*

Quando um processo é interrompido no meio de sua execução por causa de um estouro do temporizador de *quantum*, o SO precisa escalonar outro processo para ser executado e salvar o contexto deste processo interrompido. Então, alguns passos devem ser seguidos pelo escalonador na ordem em que são citados:

- Salvar o conjunto de registradores atuais do processo que foi interrompido;
- Salvar o contador de programa do processo que foi interrompido;
- Carregar os registradores do processo a ser escalonado no banco de registradores;
- Carregar o valor do contador de programa do programa a ser escalonado;
- Carregar o código do processo a ser escalonado na memória de instruções

Quando uma interrupção por *quantum* ocorre, o sistema operacional toma controle de volta a partir da linha 54 do [Algoritmo 4.16](#), onde retorna a zona de leitura e escrita para a sua própria e depois carrega o valor da interrupção. Ao verificar que a interrupção se trata do valor 0, o SO sabe que esta interrupção foi do tipo por *quantum*. Assim, a partir da linha 65, o tratamento necessário das variáveis de controle é realizado. A variável *end* recebe o valor 0 pois o processo ainda não acabou e variável *switch* recebe o valor 1 pois o processo precisa ser trocado, ou seja, escalonado. Então, o iterador *it* passa para a próxima posição, ou seja, o próximo processo, de forma circular para que os devidos procedimentos possam ser tomados.

Ao voltar para o começo do laço, o código entra na linha 16 e pula para a linha 30, onde começa a troca de contexto para outro processo. A primeira coisa a se fazer é calcular o índice do processo anterior que no caso é o processo que acabou de ser interrompido e ainda encontra-se na memória. Assim, na linha 35 o contador de programa do processo interrompido é salvo no vetor *roundPC* para uso futuro e seus registradores são salvos em seu espaço para troca de contexto. Este espaço de troca de contexto, como explicado anteriormente, é um espaço reservado na própria memória de dados do SO, onde

a região onde os registradores são salvos depende da região de memória onde estavam sendo executados. Como a memória de dados possui espaço suficiente para carregar o limite de 10 processos, os valores da memória de dados não precisam serem salvos em lugar algum, mas como existe apenas 1 banco de registradores para processos e um para o SO, o contexto do banco precisa ser salvo toda vez que um processo precisa ser escalonado. Todo o espaço de memória do SO para uso normal acaba na posição 349 e, na posição 350 em diante, está localizado a região para salvar os registradores dos processos. Para o processo que está na região número 2 por exemplo salva-se seus registradores na posição 382 em diante até que os 32 registradores estejam salvos.

Após salvar o contexto do processo interrompido, o próximo processo precisa ser carregado e executado. Para isso, nas linhas 38 a 40, carrega-se o contexto do processo. na linha 38 os registradores do processo a ser carregado são carregados da memória reservada do SO, de acordo com a região de memória reservada ao processo, de volta no banco de registradores, sobrescrevendo os valores do processo interrompido. Após isso, o código do processo é carregado do HD para a memória de instruções e o valor de seu contador de programa é carregado de volta para o PC.

Esta última parte é necessária pois o processo a ser executado pode ter sido interrompido anteriormente da mesma maneira que o processo trocado foi. Assim, o SO precisa retomar a execução a partir do momento onde este parou. Para isso, carrega-se todos os valores do banco de registradores no momento em que o processo foi interrompido e carrega-se o valor do PC no momento onde este foi interrompido.

Por fim, para que o processo possa ser de fato executado, o SO recomeça o *timer* de interrupção, cedendo o *quantum* ao processo e troca a região de escrita e leitura para a região do processo, salva no vetor *roundStatus* para assim executar a chamada de sistema *toggleProcess* e passar o controle para o processo.

4.4.5.2 Interrupção por E/S

A interrupção por entrada e saída se torna bem mais fácil de lidar do que a interrupção por *timer* pois o processo interrompido não precisa ser escalonado. No caso desta interrupção, o SO detecta a interrupção na linha 74 no caso de *input* e na linha 80 no caso de *output*.

No caso de um *input* de dados, o SO executa a rotina vista no [Algoritmo 4.17](#). Esta rotina recebe o *input* de dados do usuário pela placa e salva o valor recebido no *buffer* de transferência usando a rotina *storeBuffer*.

Algoritmo 4.17 – Rotina de Entrada de Dados

```
1 void systemInput( void ){  
2     int x;
```

```

3
4     x = input();
5     storeBuffer( x );
6 }

```

Da mesma maneira que o SO pode manipular os dados deste *buffer* de transferência, o processo em execução também pode. Assim, os dados salvos neste *buffer* podem ser acessados pelo processo de usuário.

Quando o laço recomeça na linha 14 do [Algoritmo 4.16](#), as variáveis *end* e *switch* estão com o valor 0, pois o processo não terminou e nem precisa ser trocado. Assim, nada é feito e o mesmo processo volta a executar de onde parou. Neste momento, após a chamada de sistema para *input* pelo processo de usuário, a próxima linha deve ser uma chamada da rotina *loadBuffer* onde o processo pode receber os dados de entrada que o SO salvou neste *buffer*, completando o processo de *input* por intermédio do SO.

Para o *otuput* o processo reverso é realizado. O processo de usuário salva o valor a ser mostrado no *display* no *buffer* de transferência com a rotina *storeBuffer* e faz uma chamada de sistema para *output*. Neste momento o SO entra em ação e realiza a rotina da linha 83 do [Algoritmo 4.16](#), executando os passos do [Algoritmo 4.18](#).

Algoritmo 4.18 – Rotina de Saída de Dados

```

1 void systemOutput( void ){
2     int x;
3
4     x = loadBuffer();
5     output( x );
6 }

```

Percebe-se o processo contrário nesta rotina pois o SO carrega o valor do *buffer* de transferência com a rotina *loadBuffer* e faz um *output* dos dados. Assim, o processo de *output* por intermédio do SO está completo.

4.4.5.3 Interrupção por Fim de Processo

Quando um processo termina, este precisa sinalizar ao SO através da rotina *toggleProcess* passando o valor 3 como o de interrupção, assim quando o SO retoma o controle, pode tomar as devidas providências.

Na linha 86 do [Algoritmo 4.16](#) o SO altera os valores das variáveis *end* e *switch* para 1 pois o processo atual acabou e precisa ser escalonado por outro e, além disso, caso este seja o último processo, o valor de *n* é atualizado para 0 terminando o laço do escalonador.

Quando a execução volta para a linha 16, verifica-se se o processo que acabou trata-se do último na fila e, caso seja, basta passar o iterador para o próximo processo no começo da fila circular, no caso na posição 0, e diminuir o tamanho da fila em 1, como na linha 28. Caso contrário, a fila precisa ser deslocada para que o processo possa ser deletado corretamente da fila de execução. Para isso, todos os processos a sua frente são deslocados uma posição para trás, mantendo a ordem correta da fila com a exclusão deste processo. Logo após, o iterador é passado para o próximo de forma circular nas linhas 26 e 27 e o tamanho da fila é decrementado.

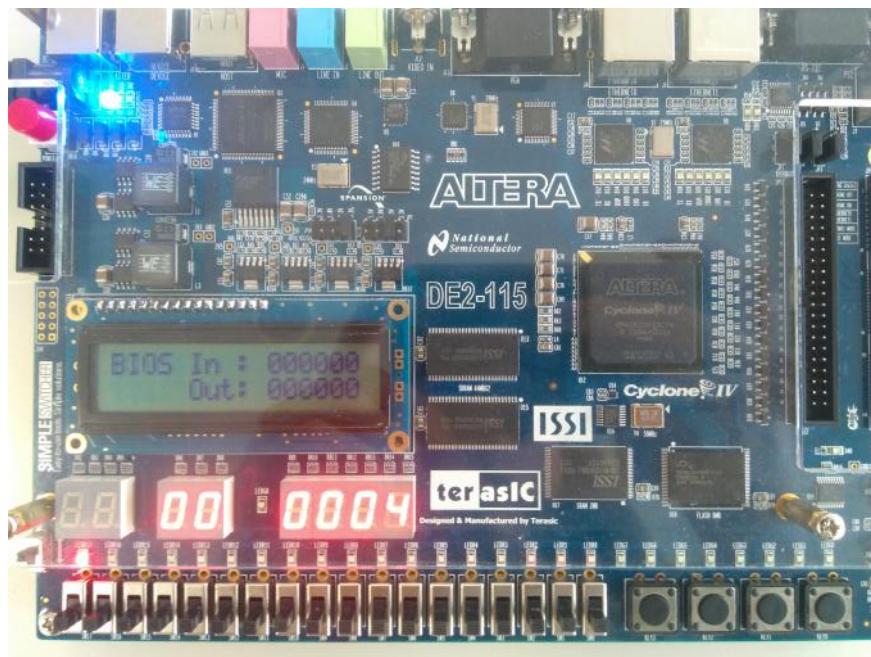
Como este processo acabou, seu contexto não precisa ser salvo, assim as linhas 31 a 36 não são executadas pois estas cuidam de salvar o contexto do processo interrompido. Assim, a partir da linha 38 até a linha 40 o próximo processo é carregado na memória e está pronto para ser executado.

5 Resultados Obtidos

Todas as funcionalidade do SO foram testadas e comprovadas. Nesta seção é possível analisar os resultados através de fotos retiradas da própria FPGA. As saídas e entradas de dados assim como as mensagens do sistema são feitas através do *display* LCD. Os 4 *displays* de 7 segmentos mais da direita indicam o contador de programa do processo em execução e os 2 segmentos do meio indicam qual processo está sendo executado.

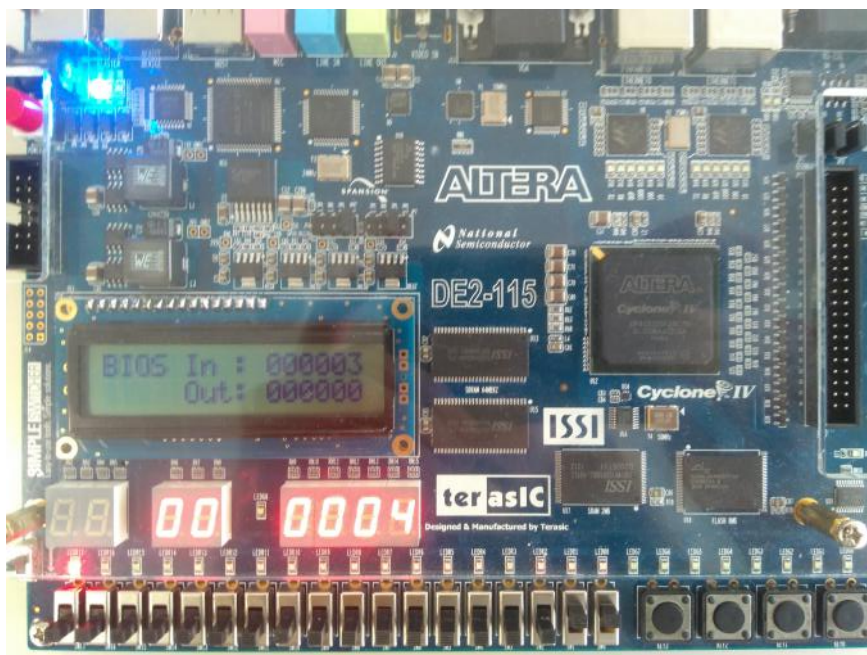
Primeiramente estão os testes da BIOS vistos na [Figura 9](#), [Figura 10](#), [Figura 11](#), [Figura 12](#) e [Figura 13](#).

Figura 9 – Teste BIOS - 1



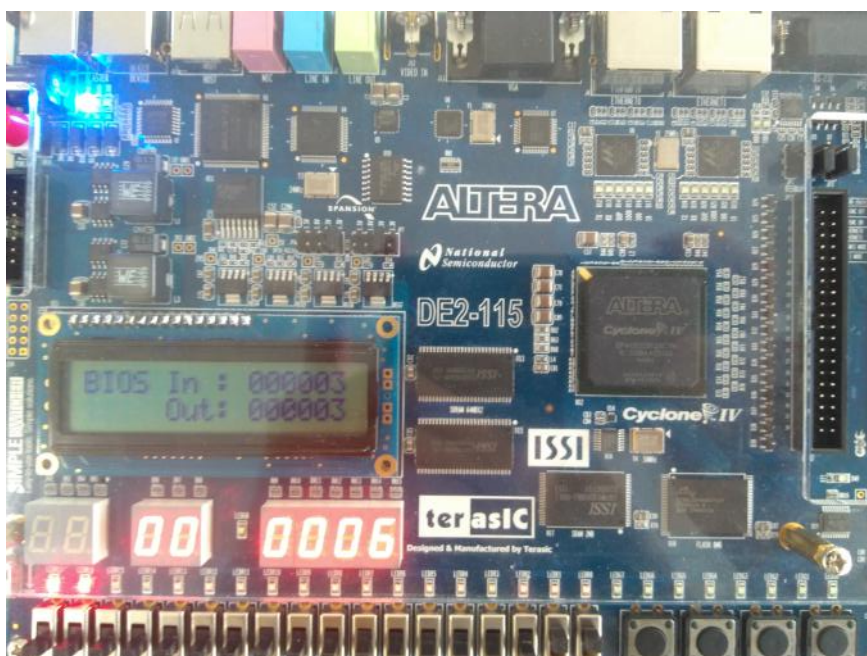
Fonte: Autoria Própria

Figura 10 – Teste BIOS - 2



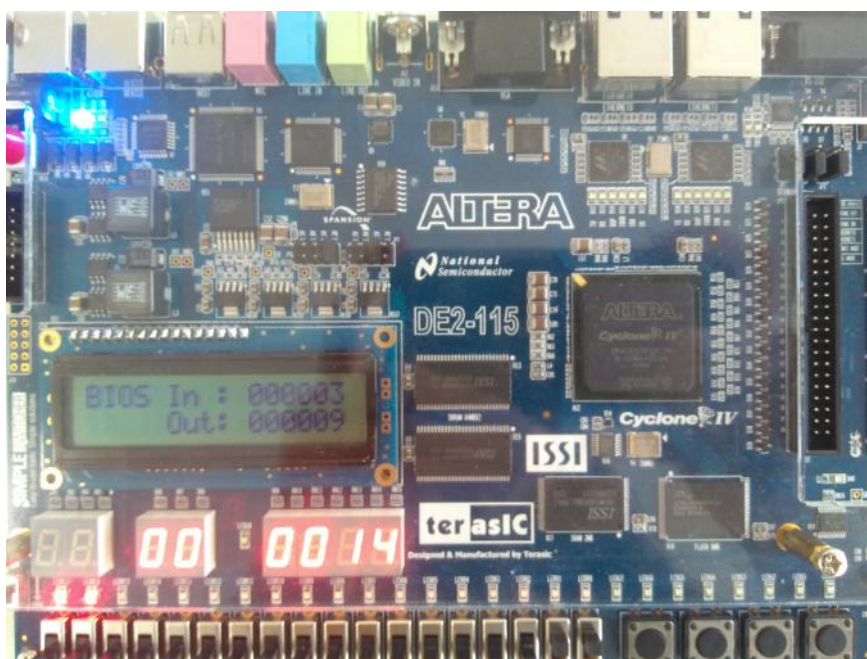
Fonte: Autoria Própria

Figura 11 – Teste BIOS - 3



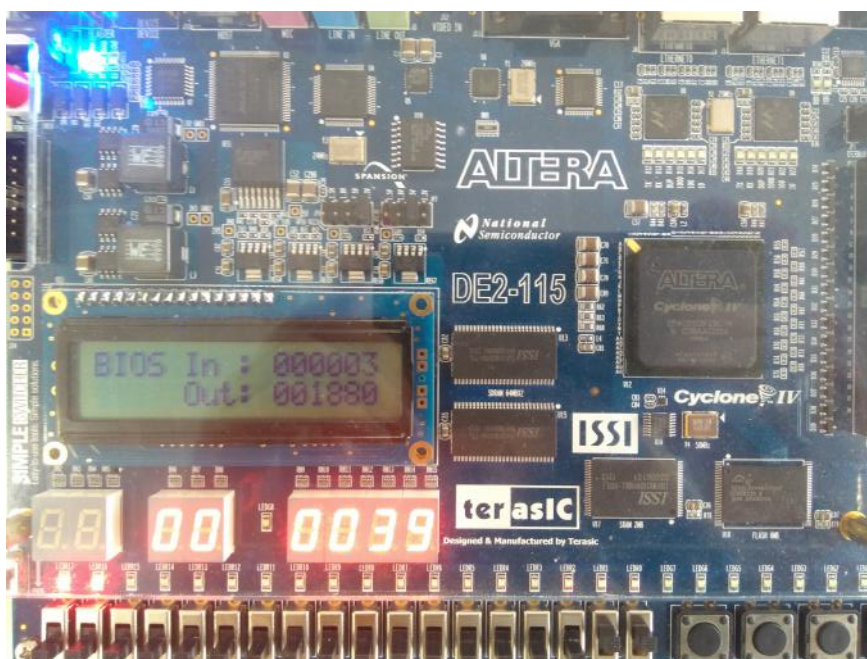
Fonte: Autoria Própria

Figura 12 – Teste BIOS - 4



Fonte: Autoria Própria

Figura 13 – Teste BIOS - 5



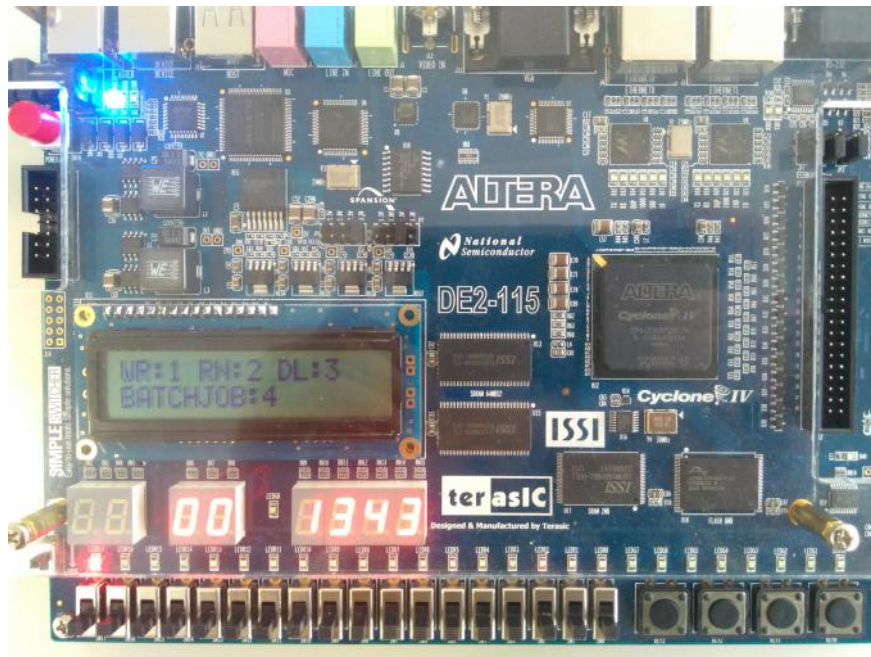
Fonte: Autoria Própria

Nas primeiras 4 figuras estão os testes de entrada, saída e manipulação de dados. Percebe-se que o dado entrado 3 é mostrado corretamente no *display* da Figura 10 na parte *In* e logo depois é mostrado como saída na Figura 11 na parte *Out*. Depois que a

multiplicação 3×3 é realizada, o resultado é mostrado na Figura 12. Na Figura 13 pode-se ver que 1880 linhas de código do SO foram copiadas do HD para a memória de instruções, completando assim a BIOS.

Na Figura 14, pode-se ver o *prompt* de comando do SO para escolher as 4 ações disponíveis: escrever, renomear, deletar e rodar processos.

Figura 14 – *Prompt* do SO

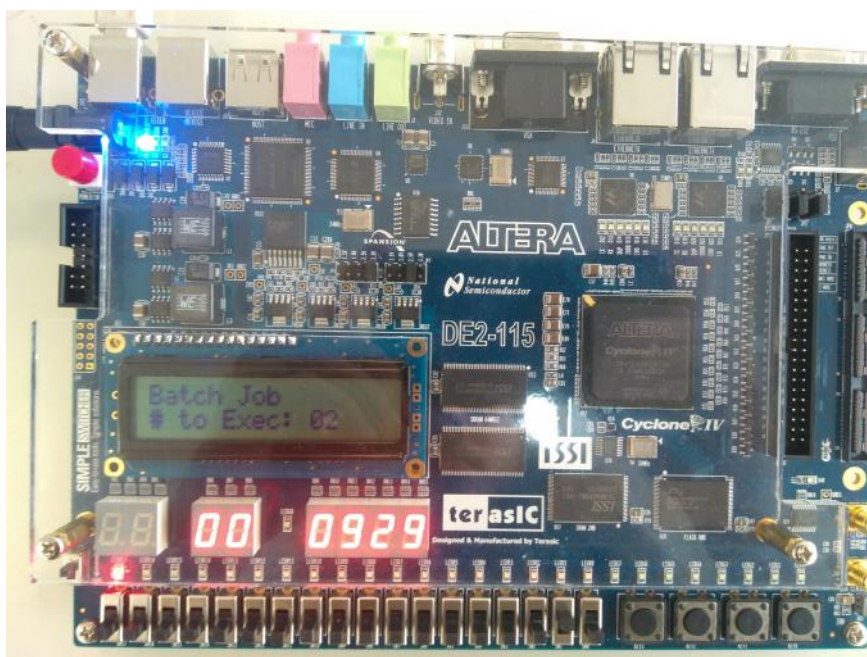


Fonte: Autoria Própria

Para a escolha de alguma das 4 funcionalidades basta o usuário entrar com o número correspondente nas chaves da placa FPGA. O procedimento realizado para testar o escalonador foi executar dois processos com preempção de *quantum* de 30 ciclos de *clock*, ambos os processos com operações de entrada e saída por intermédio do SO. Os processos executados foram a sequência de Fibonacci recursiva para o 4 número da sequência(processo de *id* 50) e o fatorial recursivo para o número de entrada do usuário(processo de *id* 51).

Os testes com processos sendo executados são feitos na Figura 15, Figura 16, Figura 17, Figura 18, Figura 19, Figura 20, Figura 21, Figura 22 e Figura 23.

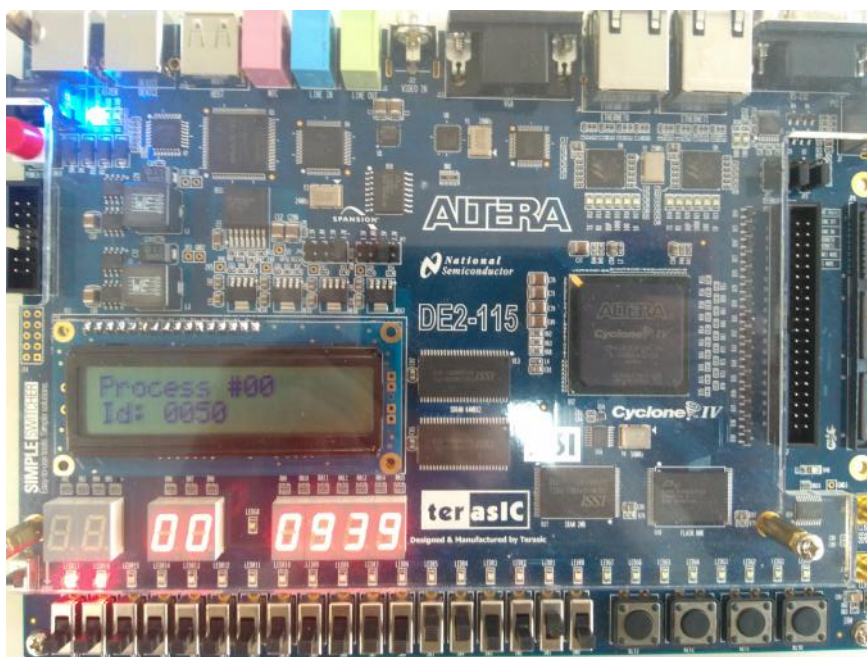
Figura 15 – Escalonador de Processos - 1



Fonte: Autoria Própria

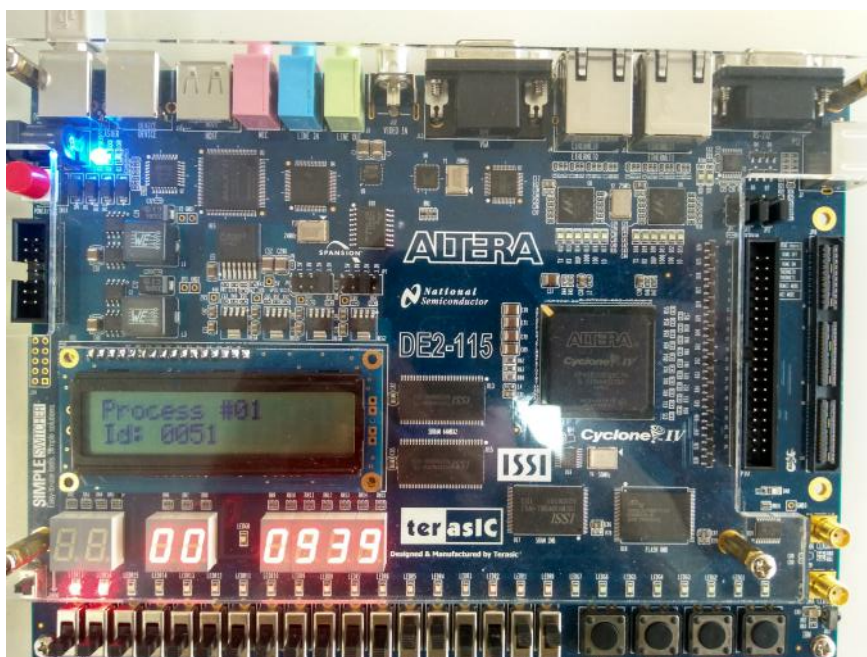
Na Figura 15, o usuário deve inserir a quantidade de processos a serem executados, neste caso 2.

Figura 16 – Escalonador de Processos - 2



Fonte: Autoria Própria

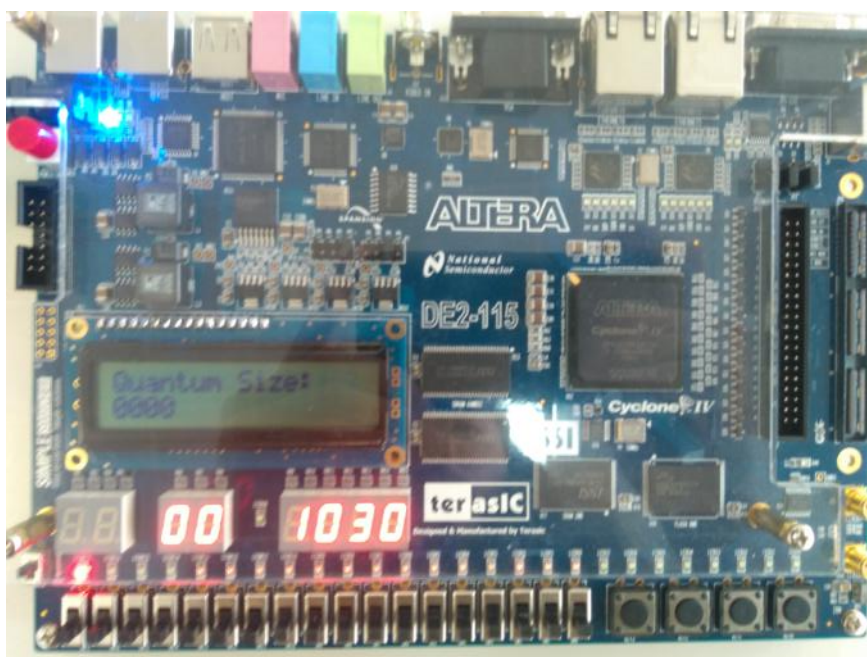
Figura 17 – Escalonador de Processos - 3



Fonte: Autoria Própria

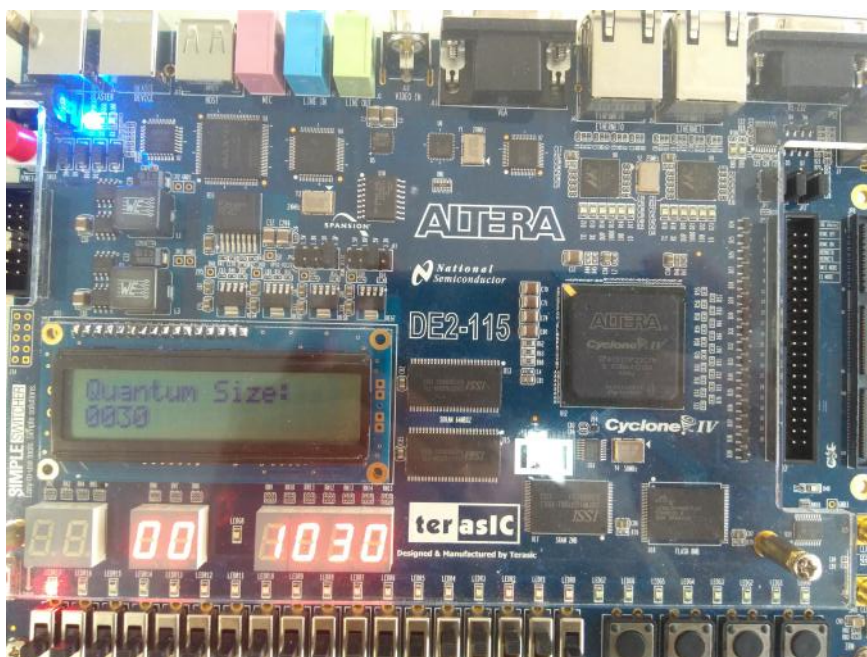
Na Figura 16 e Figura 17, o usuário deve entrar com o *id* dos processos a serem executados, no caso os processos de *id* 50 e 51.

Figura 18 – Escalonador de Processos - 4



Fonte: Autoria Própria

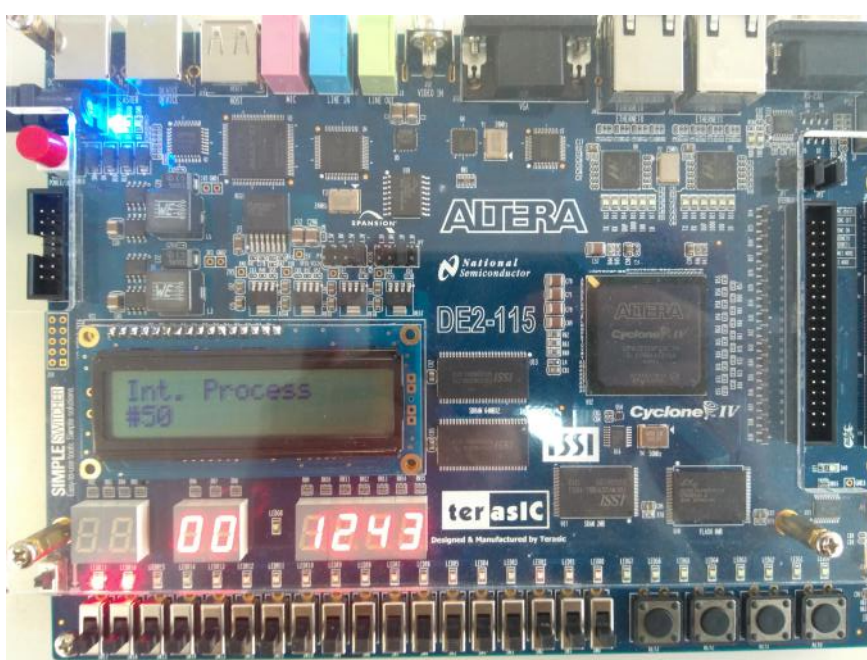
Figura 19 – Escalonador de Processos - 5



Fonte: Autoria Própria

Na Figura 18 e Figura 19, o usuário deve escolher o tamanho do *quantum* de interrupção, sendo que com a escolha de 0 não existe preempção. Neste caso a escolha da Figura 19 foi realizada e os processos são interrompidos a cada 30 ciclos de *clock*.

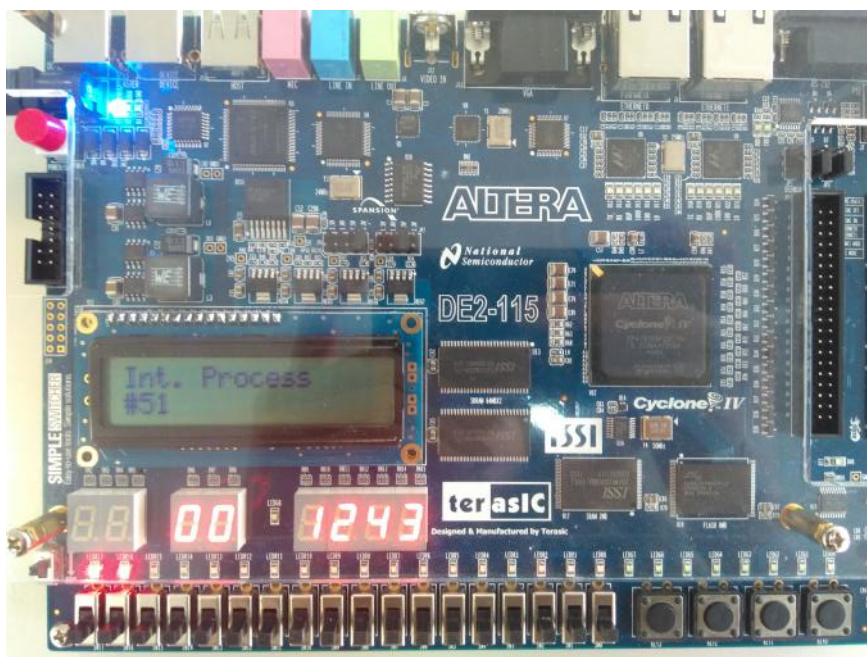
Figura 20 – Escalonador de Processos - 6



Fonte: Autoria Própria

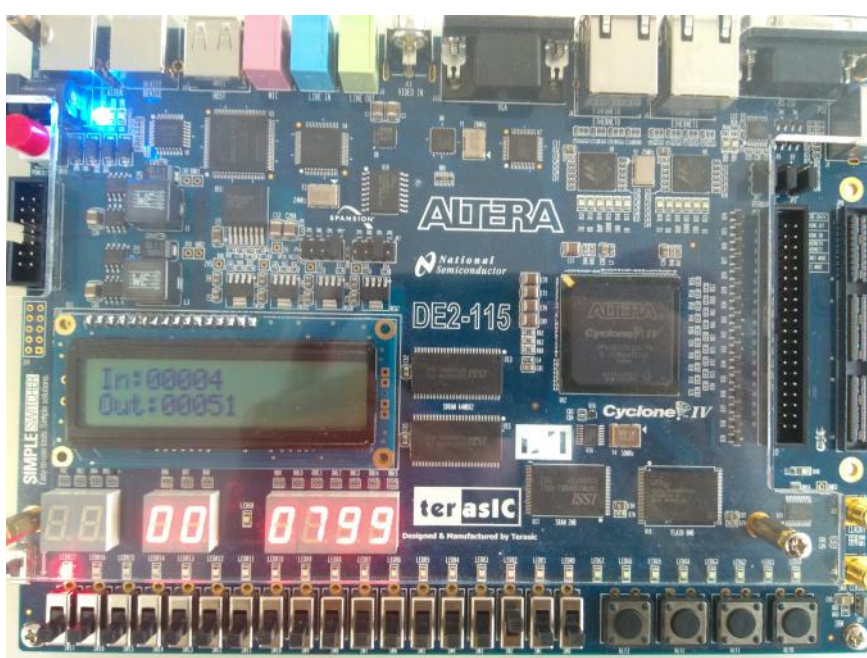
Na Figura 20 o primeiro processo, o Fibonacci, é interrompido por estouro de temporizador e vai ser substituído pelo processo 51. A mensagem no LCD mostra o *id* do processo interrompido.

Figura 21 – Escalonador de Processos - 7



Fonte: Autoria Própria

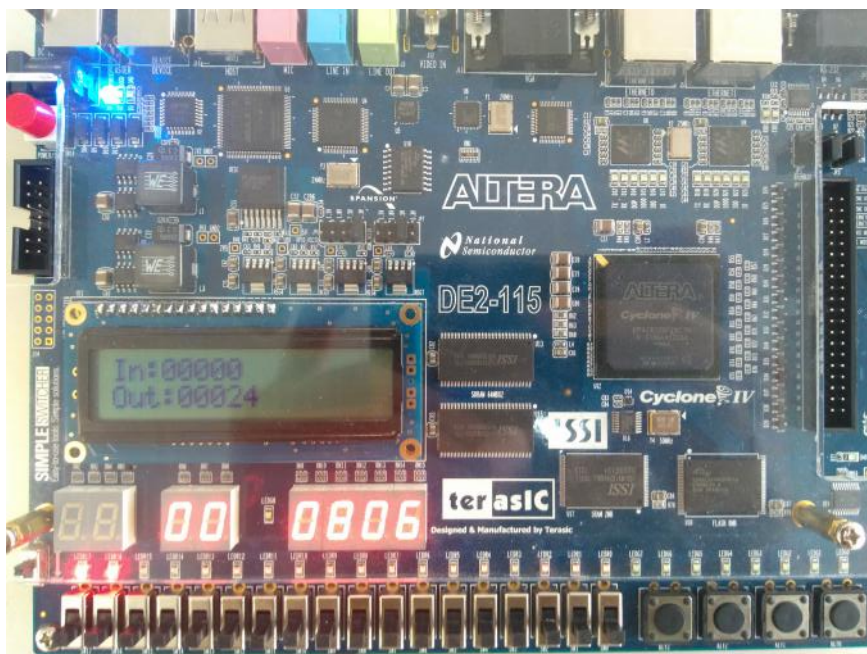
Figura 22 – Escalonador de Processos - 8



Fonte: Autoria Própria

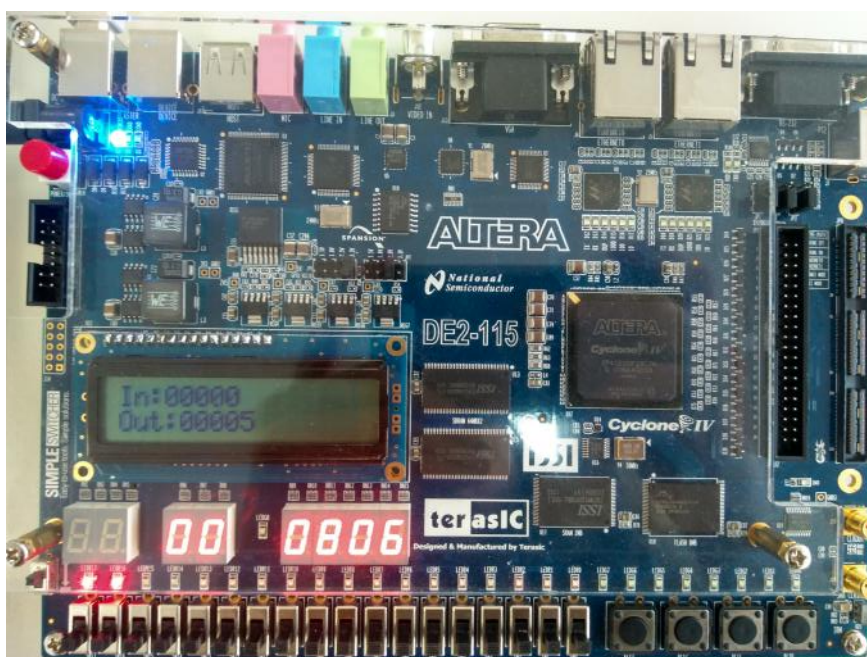
Na Figura 21 e Figura 22, o processo 51 é interrompido para realizar a entrada de dados. O valor entrado então é 4 e o processos 51 deve calcular o fatorial recursivo de 4.

Figura 23 – Escalonador de Processos - 9



Fonte: Autoria Própria

Figura 24 – Escalonador de Processos - 10

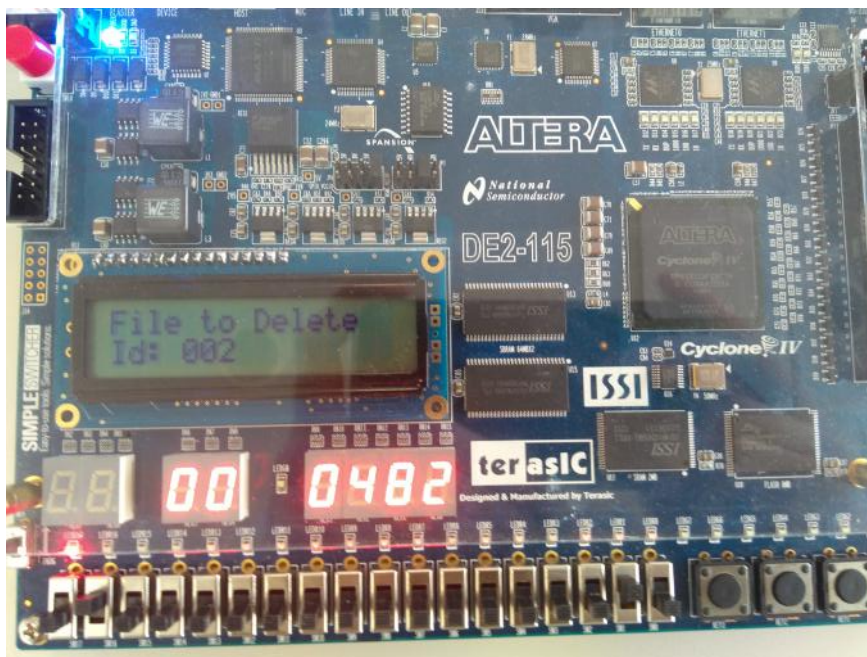


Fonte: Autoria Própria

Na Figura 23 e Figura 24, os resultados dos processos 51 e 50 são gerados como *output* do SO respectivamente, e verifica-se de que o o fatorial de 4 resultou em 24 corretamente assim como o quarto número de Fibonacci resultou em 5 como esperado

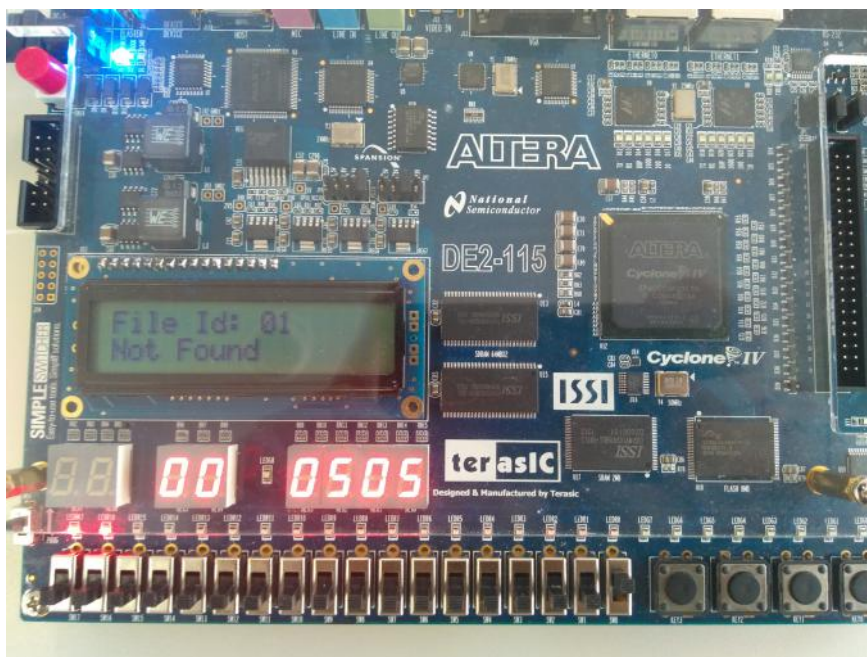
Passando para o gerenciamento de arquivos, primeiramente analisa-se os testes com deleção de arquivos. Na Figura 25 e Figura 26, respectivamente, o arquivo

Figura 25 – Deletar Arquivos - 1



Fonte: Autoria Própria

Figura 26 – Deletar Arquivos - 2

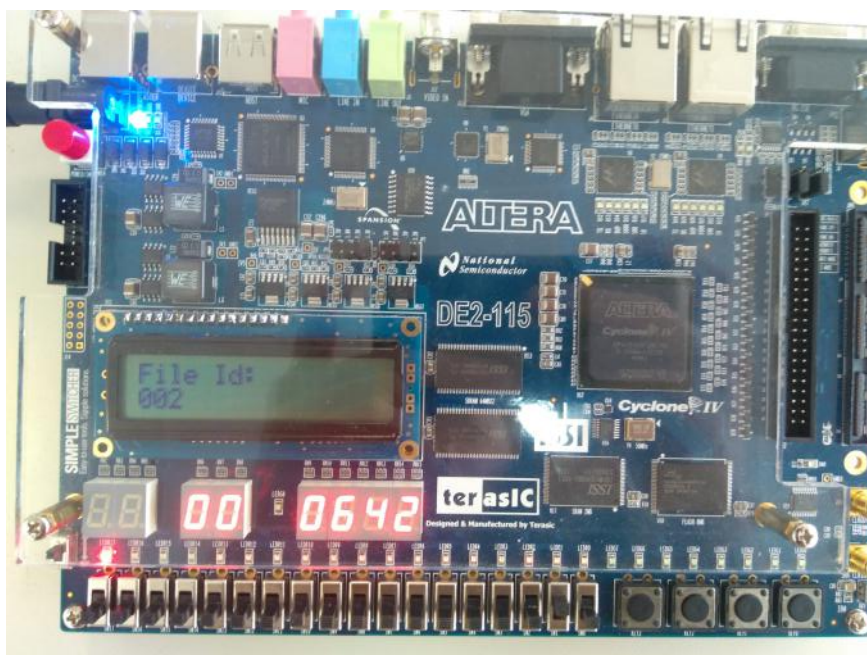


Fonte: Autoria Própria

Na [Figura 25](#), o arquivo de *id* é selecionado para ser deletado. Posteriormente, na [Figura 26](#), o arquivo de *id* 1 é selecionado para ser deletado, mas como este não existe, a mensagem de erro desta imagem é vista.

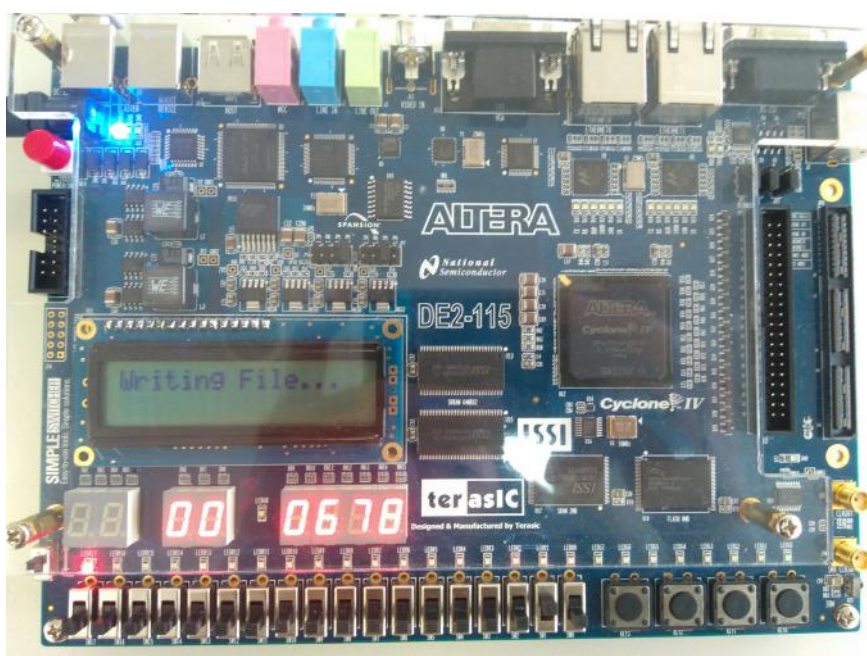
Seguindo, o processo de escrita de arquivos é visto na [Figura 27](#), [Figura 28](#) e [Figura 29](#).

Figura 27 – Escrever Arquivos - 1



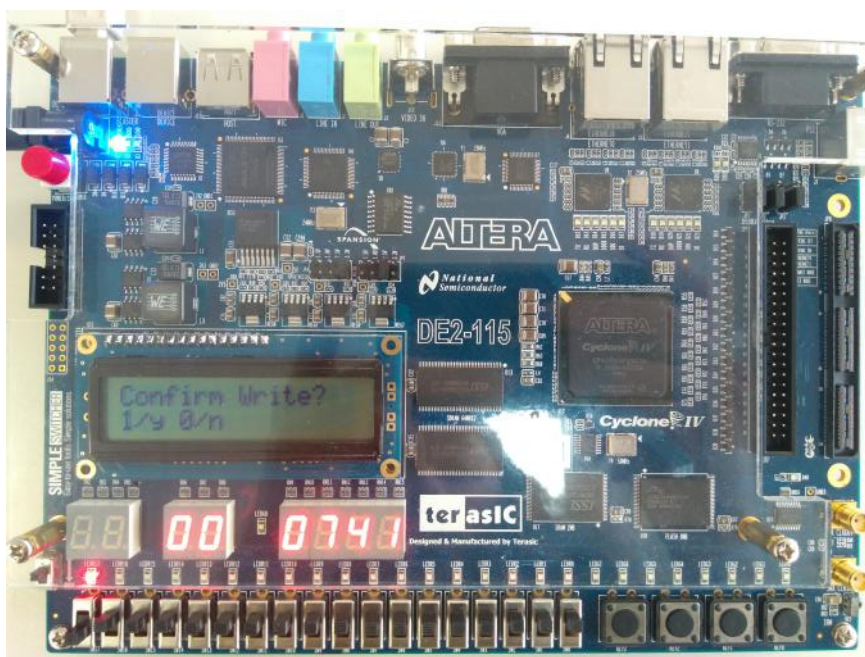
Fonte: Autoria Própria

Figura 28 – Escrever Arquivos - 2



Fonte: Autoria Própria

Figura 29 – Escrever Arquivos - 4



Fonte: Autoria Própria

Na Figura 27, o usuário deve entrar com o *id* do arquivo a ser escrito, neste caso o *id* 2. Na Figura 28, começa o laço de escrita dos dados. Neste laço o usuário entra com todas os dados a serem escritos até que decida acabar a escrita. Com a escrita finalizada na Figura 29, o usuário deve confirmar escrita inserindo o valor 1 declinando a escrita do arquivo inserindo 0.

6 Considerações Finais

O sistema computacional desenvolvido nesta disciplina precisou de revisões extensas na arquitetura do processador para o qual foi desenvolvido para que todo o suporte para um sistema operacional funcional fosse criado. Não tão distante, o compilador para C- previamente desenvolvido também sofreu mudanças mas não tão dramáticas quanto o anterior. Diversos módulos novos em *hardware* foram desenvolvidos e muitos módulos já existentes tiveram de ser mudados para a realização deste projeto.

De certa forma o desenvolvimento deste projeto foi exaustivo e trabalhoso, mas com a base teórica e prática adquirida até agora pode-se terminar o projeto de forma que os resultados foram os esperados e refletem o trabalho empenhado para a conclusão do mesmo.

De certa forma o sistema está completo de acordo com as diretrizes da disciplina, mas busca-se melhorar o sistema de comunicação entre o usuário pelo LCD e principalmente a entrada de dados, onde a ideia é utilizar alguma forma de comunicação serial. Além disso, otimizações no código do SO podem ser feitas para melhor compreensão futura.

Por fim, este projeto colocou em prática todos os conhecimentos sobre arquitetura de computadores, compiladores, sistemas operacionais e *Verilog* em prática para que todas as partes fossem integradas entre si de forma que este sistema computacional funcionasse de forma correta e precisa.

Referências

- 1 TANENBAUM, A. S.; FILHO, N. M. *Sistemas operacionais modernos*. [S.l.]: Prentice-Hall, 1995. v. 3. Citado 9 vezes nas páginas 10, 11, 12, 13, 14, 15, 16, 17 e 18.
- 2 KIMURA, B. Slides disciplina sistemas operacionais. UNIFESP. 2018. Citado 3 vezes nas páginas 10, 11 e 17.
- 3 PATTERSON, D. A.; HENNESY, J. L. *Computer Organization and Design*. 5th edition. ed. Waltham/MA, EUA: Morgan Kaufmann, 2007. Citado na página 13.
- 4 KOZIEROK, C. M. *System BIOS*. 1997–2004. <<http://www.pcguide.com/ref/mbsys/bios/index.htm>>. [Online, acessado 22/09/2018]. Citado na página 18.
- 5 LORENA, A. C. *Aulas da Disciplina Compiladores*. 2017. [Online, acessado via moodle]. Citado na página 20.

Apêndices

APÊNDICE A – Apêndice 1

Algoritmo A.1 – Código do SO em C-

```
1  int  INTERRUPTFLAG;
2  int  FILEID;
3
4  int  fileLocation[35];
5  int  fileStart[35];
6  int  fileEnd[35];
7  int  fileId[35];
8  int  fileExists[35];
9  int  fileSize[35];
10 int  fileCounter;
11 int  trackCounter;
12 int  filesStart;
13
14 int  roundQueue[10];
15 int  roundStatus[10];
16 int  roundPC[10];
17
18 int  OSSTART;
19 int  FWRITEA;
20 int  FWRITEB;
21 int  FWRITEC;
22 int  FRENAME;
23 int  FDELETE;
24 int  BATCHJOB;
25 int  PROCESSRUN;
26 int  PSELECT;
27 int  IPROCESS;
28 int  NPROCESS;
29 int  FCONFIRM;
30 int  FNFND;
31 int  FAEXT;
32 int  TIMER;
33 int  MAXSECTOR;
34 int  MAXTRACKS;
```

```
35
36 int nProcess;
37
38 void startHD( void ){
39
40     int i; int k; int regs;
41
42     fileLocation[0] = 0;
43     fileStart[0] = 0;
44     fileEnd[0] = 1879;
45     fileId[0] = 0;
46     fileExists[0] = 1;
47     fileSize[0] = 1880;
48
49     fileCounter = 1;
50     trackCounter = 1;
51
52     i = 1;
53     k = 1880;
54     regs = 32;
55     filesStart = fileCounter;
56
57     fileLocation[fileCounter] = 1;
58     fileStart[fileCounter] = 0;
59     fileEnd[fileCounter] = 70;
60     fileId[fileCounter] = 50;
61     fileExists[fileCounter] = 1;
62     fileSize[fileCounter] = 71;
63     fileCounter = fileCounter + 1;
64
65     fileLocation[fileCounter] = 1;
66     fileStart[fileCounter] = 71;
67     fileEnd[fileCounter] = 135;
68     fileId[fileCounter] = 51;
69     fileExists[fileCounter] = 1;
70     fileSize[fileCounter] = 65;
71     fileCounter = fileCounter + 1;
72
73     fileLocation[fileCounter] = 1;
```



```
74     fileStart[fileCounter] = 136;
75     fileEnd[fileCounter] = 189;
76     fileId[fileCounter] = 52;
77     fileExists[fileCounter] = 1;
78     fileSize[fileCounter] = 54;
79     fileCounter = fileCounter + 1;
80
81     fileLocation[fileCounter] = 1;
82     fileStart[fileCounter] = 190;
83     fileEnd[fileCounter] = 281;
84     fileId[fileCounter] = 53;
85     fileExists[fileCounter] = 1;
86     fileSize[fileCounter] = 92;
87     fileCounter = fileCounter + 1;
88
89     fileLocation[fileCounter] = 1;
90     fileStart[fileCounter] = 282;
91     fileEnd[fileCounter] = 375;
92     fileId[fileCounter] = 54;
93     fileExists[fileCounter] = 1;
94     fileSize[fileCounter] = 94;
95     fileCounter = fileCounter + 1;
96
97     fileLocation[fileCounter] = 1;
98     fileStart[fileCounter] = 376;
99     fileEnd[fileCounter] = 469;
100    fileId[fileCounter] = 55;
101    fileExists[fileCounter] = 1;
102    fileSize[fileCounter] = 94;
103    fileCounter = fileCounter + 1;
104
105    fileLocation[fileCounter] = 1;
106    fileStart[fileCounter] = 470;
107    fileEnd[fileCounter] = 542;
108    fileId[fileCounter] = 56;
109    fileExists[fileCounter] = 1;
110    fileSize[fileCounter] = 73;
111    fileCounter = fileCounter + 1;
112
```

```
113     fileLocation[fileCounter] = 1;
114     fileStart[fileCounter] = 543;
115     fileEnd[fileCounter] = 617;
116     fileId[fileCounter] = 57;
117     fileExists[fileCounter] = 1;
118     fileSize[fileCounter] = 75;
119     fileCounter = fileCounter + 1;
120
121     fileLocation[fileCounter] = 1;
122     fileStart[fileCounter] = 618;
123     fileEnd[fileCounter] = 712;
124     fileId[fileCounter] = 58;
125     fileExists[fileCounter] = 1;
126     fileSize[fileCounter] = 95;
127     fileCounter = fileCounter + 1;
128
129     fileLocation[fileCounter] = 1;
130     fileStart[fileCounter] = 713;
131     fileEnd[fileCounter] = 760;
132     fileId[fileCounter] = 59;
133     fileExists[fileCounter] = 1;
134     fileSize[fileCounter] = 48;
135     fileCounter = fileCounter + 1;
136 }
137
138 void startSystem( void ){
139     OSSTART = 1;
140     FWRITEA = 2;
141     FWRITEB = 3;
142     FWRITEC = 4;
143     FRENAME = 5;
144     FDELETE = 6;
145     BATCHJOB = 7;
146     PROCESSRUN = 8;
147     PSELECT = 9;
148     IPROCESS = 10;
149     NPROCESS = 11;
150     FCONFIRM = 12;
151     FNFND = 13;
```

```
152     FAEXT = 14;
153     TIMER = 15;
154     MAXSECTOR = 2200;
155     MAXTRACKS = 2;
156     showPrompt( OSSTART );
157     startHD();
158 }
159
160 void setBatchJob( void ){
161     return;
162 }
163
164 int searchFile( int id ){
165     int i; int ret;
166
167     i = 0;
168
169     while( i < fileCounter ){
170         if( fileId[i] == id ){
171             return i;
172         }
173         i = i + 1;
174     }
175     return 0;
176 }
177
178 void renameFile( void ){
179     int i; int flag;
180     int id; int newId;
181
182     i = 0;
183     flag = 99;
184     showPrompt( FRENAME );
185     id = input();
186     newId = input();
187
188     while( i < fileCounter ){
189         if( fileExists[i] == 0 )
190             i = 99;
```

```
191     else if( fileId[i] == id ){
192         flag = i;
193         i = 99;
194     }
195     i = i + 1;
196 }
197 if( flag < fileCounter ){
198     fileId[flag] = newId;
199 }
200 else{
201     output(FNFND);
202     output(id);
203 }
204 }
205
206 void deleteFile( void ){
207     int i; int control;
208     showPrompt( FDELETE );
209     FILEID = input();
210     i = filesStart;
211     control = 1;
212     while( control != 0 ){
213         if( fileExists[i] == 0 ){
214             control = 0;
215             showPrompt(FNFND);
216             output(FILEID);
217         }
218         else{
219             if( fileId[i] == FILEID ){
220                 fileExists[i] = 0;
221                 control = 0;
222                 if( fileCounter != i ){
223                     while( fileExists[i+1] != 0 ){
224                         fileLocation[i] = fileLocation[i+1];
225                         fileStart[i] = fileStart[i+1];
226                         fileEnd[i] = fileEnd[i+1];
227                         fileId[i] = fileId[i+1];
228                         fileExists[i] = fileExists[i+1];
229                         fileSize[i] = fileSize[i+1];
```

```
230         i = i + 1;
231     }
232     fileCounter = fileCounter - 1;
233 }
234 }
235     i = i + 1;
236 }
237 }
238 }
239
240 void writeFile(void){
241     int i; int instruction; int id; int aux;
242     int sa; int sb; int control;
243     int wLoc; int track;
244
245     control = 0;
246     sa = 0;
247     sb = 0;
248     wLoc = fileEnd[fileCounter-1];
249     track = trackCounter;
250
251     control = 0;
252     showPrompt( FWRITEA );
253     id = input();
254     aux = searchFile(id);
255     if( aux != 0 ){
256         showPrompt(FAEXT);
257         output(id);
258         return;
259     }
260     while( control != (0-1)){
261         showPrompt( FWRITEB );
262         sa = input();
263         sb = input();
264         instruction = shiftBits( sa, sb );
265         if( wLoc == MAXSECTOR ){
266             wLoc = 0;
267             track = track + 1;
268         }
```

```
269     else{
270         wLoc = wLoc + 1;
271     }
272     storeBuffer( instruction );
273     cpyMemToHD( track, wLoc );
274     i = i + 1;
275     output( i );
276     showPrompt( FWRITEC );
277     control = input();
278 }
279 if( i != 0 ){
280     showPrompt( FCONFIRM );
281     control = input();
282     if( control == 1 ){
283         fileLocation[fileCounter] = trackCounter;
284         fileStart[fileCounter] = fileEnd[fileCounter-1]+1;
285         fileEnd[fileCounter] = wLoc-1;
286         fileId[fileCounter] = id;
287         fileExists[fileCounter] = 1;
288         fileSize[fileCounter] = i;
289         fileCounter = fileCounter + 1;
290         trackCounter = track;
291     }
292 }
293 }
294
295 void systemInput( void ){
296     int x;
297
298     x = input();
299     storeBuffer( x );
300 }
301
302 void systemOutput( void ){
303     int x;
304
305     x = loadBuffer();
306     output( x );
307 }
```

```
308
309 void loadCode( int id ){
310     int i; int it; int t; int s;
311
312     i = searchFile( id );
313
314     if( i != 0 ){
315         s = 0;
316         it = fileStart[i];
317         t = fileLocation[i];
318         nop();
319
320         while( s < fileSize[i] ){
321             cpyHDTtoInstr( 1, t, it );
322             if( it == MAXSECTOR ){
323                 it = 0;
324                 t = t + 1;
325             }
326             else{
327                 it = it + 1;
328             }
329             s = s + 1;
330         }
331     }
332 }
333
334 void loadRegisters( int p ){
335     loadRegs( p );
336 }
337
338 void storeRegisters( int p ){
339     saveRegs( p );
340 }
341
342 void startVets( void ){
343     int i; int it; int aux;
344     it = 0;
345
346     showPrompt( BATCHJOB );
```



```
347     nProcess = input();
348
349     while( it < nProcess ){
350         showPrompt( PSELECT );
351         output( it );
352         i = input();
353         aux = searchFile(i);
354         if( aux != 0 ){
355             roundQueue[it] = i;
356             roundStatus[it] = it+1;
357             roundPC[it] = 0;
358             it = it + 1;
359         }
360         else{
361             showPrompt(FNFND);
362             output(i);
363         }
364     }
365     while( it < 10 ){
366         roundQueue[it] = 0;
367         roundStatus[it] = 0;
368         roundPC[it] = 0;
369         it = it + 1;
370     }
371     showPrompt( PROCESSRUN );
372 }
373
374 void roundRobin( void ){
375     int it; int i; int n; int switch; int k; int end; int ant;
376     int p;
377
378     it = 0;
379     switch = 1;
380     end = 0;
381
382     startVets();
383     showPrompt(TIMER);
384     i = input();
385     setTimer( i );
```

```
385     n = nProcess;
386     savePC(0);
387     while( n > 0 ){
388         if( switch != 0 ){
389             if( end == 1 ){
390                 if( it < n-1 ){
391                     k = it;
392                     while( k < n-1 ){
393                         roundPC[k] = roundPC[k+1];
394                         roundQueue[k] = roundQueue[k+1];
395                         roundStatus[k] = roundStatus[k+1];
396                         k = k + 1;
397                     }
398                 }
399                 if( it == n - 1 )
400                     it = 0;
401                 n = n - 1;
402             }
403             else{
404                 if( it == 0 )
405                     ant = n-1;
406                 else
407                     ant = it - 1;
408                 roundPC[ant] = loadPC();
409                 storeRegisters( roundStatus[ant]-1 );
410             }
411             loadRegisters( roundStatus[it] -1 );
412             loadCode( roundQueue[it] );
413             savePC( roundPC[it] );
414             output( roundPC[it] );
415         }
416
417         p = roundStatus[it];
418
419         if( INTERRUPTFLAG == 0 )
420             resetTimer();
421         if( INTERRUPTFLAG == 3 )
422             resetTimer();
423     }
```

```
424     setWriteProcess(p);
425     setReadProcess(p);
426     toggleProcess(0);
427     setWriteProcess(0);
428     setReadProcess(0);
429     INTERRUPTFLAG = loadInterruption();
430
431     k = roundQueue[it];
432
433     showPrompt( IPROCESS );
434     output( k );
435     showPrompt( PROCESSRUN );
436
437     /* Timer */
438     if( INTERRUPTFLAG == 0 ){
439         end = 0;
440         switch = 1;
441         if( it == n - 1 )
442             it = 0;
443         else
444             it = it + 1;
445     }
446     /* Input */
447     else if( INTERRUPTFLAG == 1 ){
448         end = 0;
449         switch = 0;
450         systemInput();
451     }
452     /* Output */
453     else if( INTERRUPTFLAG == 2 ){
454         end = 0;
455         switch = 0;
456         systemOutput();
457     }
458     /* End */
459     else if( INTERRUPTFLAG == 3 ){
460         end = 1;
461         switch = 1;
462         if( n == 1 )
```

```
463         n = 0;
464     }
465 }
466 }
467
468 void main(void){
469     int i;
470     int file;
471     i = 10;
472
473     showPrompt( OSSTART );
474     startSystem();
475     while( i != 0 ){
476         showPrompt( OSSTART );
477         i = input();
478         if( i == 1 ){
479             writeFile();
480         }
481         else if( i == 2 ){
482             renameFile();
483         }
484         else if( i == 3 ){
485             deleteFile();
486         }
487         else if( i == 4 ){
488             roundRobin();
489         }
490     }
491 }
```